

مرور جامع بر اساس داکیومنت رسمی — سپتامبر ۲۰۲۶

React 19 + Next.js 16

مرور صفر تا مصاحبه

فصل‌بندی مطابق react.dev و nextjs.org/docs — با مثال، تمرین و

جواب

ISR & Caching

Server Actions

Server Components

Deployment

React 19 Hooks

Streaming & Suspense

۱۲ فصل در دو بخش · چیت‌شیت دوتایی · ۲۰۰ سوال مصاحبه با جواب

فهرست مطالب

نقشه مرور و روش استفاده	شروع مرور	1
کامپوننت خالص، props، key، تله &&	فصل ۱ – توصیف UI	2
snapshot، immutable، reducer، Context	فصل ۲ – مدیریت State	3
useRef، useEffect، custom hooks	فصل ۳ – Effects و Refs	4
use، useActionState، useOptimistic	فصل ۴ – React 19 مدرن	5
layout، Link، قرارداد فایل‌ها	فصل ۵ – ساختار و Routing	6
مرز serialization، RSC، ترکیب	فصل ۶ – Server vs Client ★	7
SSG/ISR/SSR، caching، Promise.all	فصل ۷ – Fetching و رندر	8
zod، revalidate، فرم‌ها	فصل ۸ – Server Actions	9
Suspense، loading/error/not-found	فصل ۹ – Streaming و ارورها	10
Image، Font، Metadata، sitemap	فصل ۱۰ – Image/Font/Metadata	11
route.ts، middleware، env، deploy	فصل ۱۱ – API، Middleware، Deploy	12
چیت‌شیت دوتایی + ۲۰ سوال مصاحبه	فصل ۱۲ – چیت‌شیت و مصاحبه	13
	فصل ۱۳ – عمق‌سنجی ۹ سوال کلیدی	14

مرور جامع React و Next.js – بر اساس داکيومنت رسمی

نسخه دوره: سپتامبر ۲۰۲۶ – مرور بر اساس React 19.2 و Next.js 16.3 ساختار: فصل‌بندی دقیقاً پیرو داکيومنت رسمی – react.dev (بخش Learn و API Reference) و (App Router) nextjs.org/docs – با لینک منبع هر فصل مخاطب: تو که با Next.js کار می‌کنی و می‌خواهی دانشات را مرتب، کامل و بدون حفره کنی – به‌ویژه برای مصاحبه

این مرور چه فرقی با داکيومنت خواندن دارد؟

داکيومنت رسمی انگلیسی، پراکنده و طولانی است. این دوره: - **فشرده و فارسی** – هر فصل = یک بخش داکيومنت، خلاصه و نکته‌محور - **با مقایسه و تله‌ها** – چیزهایی که داکيومنت می‌گوید ولی در عمل ازشان خطا می‌رود - **تمرین با جواب** – برای اینکه «خوانده» نشوی، «بلد» شوی - **لینک منبع رسمی هر فصل** – برای عمیق‌شدن هرچا خواستی

نقشه دوره

بخش اول – React 19 (react.dev)

فصل	موضوع	معادل داکيومنت
فصل ۱	توصیف UI	Describing the UI
فصل ۲	مدیریت State	Managing State
فصل ۳	Escape Hatches	Refs, Effects, Custom Hooks
فصل ۴	React 19 و پترن‌های مدرن	Reference: use, useActionState, useOptimistic و ...

فصل	موضوع	معادل داکيومنت
فصل ۵	ساختار و Routing	Project Structure, Layouts & Pages, Linking
فصل ۶	★ Server vs Client Components	Server and Client Components
فصل ۷	Fetching و استراتژی رندر	Data Fetching, Caching, Static/Dynamic
فصل ۸	Mutations و Server Actions	Updating Data, Forms
فصل ۹	Streaming و مدیریت ارور	Streaming, loading/error/not-found
فصل ۱۰	بهینه‌سازی	Images, Fonts, Metadata
فصل ۱۱	Route Handlers, Middleware, Deploy	Route Handlers, Middleware, Deploying
فصل ۱۲	چیت‌شیت دوتایی + گلاساری + مصاحبه	—
فصل ۱۳	عمق‌سنجی: use, useActionState, forwardRef, Edge, RSC Payload, Hydration, RPC	—

نحوه استفاده از این مرور

- مرور سریع (یک روز): فصل ۱، ۲، ۶، ۷، ۸ – قلب مصاحبه‌ها
 - مرور کامل (یک هفته): هر فصل را بخوان، تمرین‌هایش را در یک sandbox حل کن، جواب‌ها را مقایسه کن
 - رفرنس همیشگی: چیت‌شیت‌های فصل ۱۲ کنار مانیتورت
- هر فصل با لینک داکيومنت رسمی شروع می‌شود – اگر جایی ابهام داشتی، منبعش همیشه یک کلیک دور است.

پیش‌نیاز

تجربه عملی با Next.js (که داری – پنج دوره قبلی پرش کرده بودی!) + TS. این دوره «آموزش از صفر» نیست؛ مرور منظم و تکمیل حفره‌هاست.

نسخه PDF رنگی راست به چپ (فونت وزیر متن) در همین پوشه: «**مرور کامل React و Next.js.pdf**». بازسازی: `python build/build_pdf.py` + رندر Chrome headless.

فصل ۱ – توصیف (Describing the UI)

منبع رسمی: react.dev/learn – بخش Describing the UI 🎯 هدف: مفاهیم پایه‌ای که همه «بلدیم» ولی در مصاحبه دقیق توضیحشان نمی‌دهیم: کامپوننت خالص، props، keys و قواعد JSX.

۱.۱ – کامپوننت: تابعی که JSX برمی‌گرداند

```
// ✅ می‌بیند HTML آن را تگ JSX اجباری - وگرنه نام با حروف بزرگ
function Avatar({ src, size = 40 }: { src: string; size?: number }) {
  return <img src={src} width={size} height={size} alt="" />;
}

// HTML استفاده مثل تگ:
<Avatar src="/me.png" size={64} />
```

سه قانون داکيومنت: 1. **خالص بودن (Purity)**: کامپوننت نباید ورودی‌هایش را تغییر دهد یا متغیرهای بیرونی را دستکاری کند – ورودی یکسان = خروجی یکسان 2. یک کامپوننت = یک وظیفه؛ بزرگ شد بشکنش 3. فایل می‌تواند چند کامپوننت داشته باشد ولی قرارداد: یک کامپوننت اصلی هم‌نام فایل

```
// ❌ را تغییر می‌دهد prop - ناخالص
function Badge({ count }: { count: number }) {
  count = count + 1; // mutation!
  return <span>{count}</span>;
}

// ✅ خالص - از ورودی، مقدار جدید حساب می‌کند
function Badge({ count }: { count: number }) {
  return <span>{count + 1}</span>;
}
```

۱.۲ – JSX: قواعدی که گاهی فراموش می‌شود

مثال	قاعده
<code><>۱۲</></code>	یک root element (یا Fragment)
<code></code>	همه تگ‌ها بسته
<code>className</code> ، <code>onClick</code> ، <code>tabIndex</code>	تقریباً همه attribute ها camelCase
<code>{user.name}</code> ، <code>{isAdmin} && <Admin/></code>	<code>{}</code> = ورود به دنیای JS
<code>style={{ marginTop: 8 }}</code>	استایل = آبجکت
<code>{/* مثل این */}</code>	کامنت داخل JSX

Conditional rendering – چهار الگوی دایومنت:

```
{isPacked && "✅"} // && - اگر true
{isPacked ? "✅" : "❌"} // ternary - دو حالت
{error !== null && <ErrorBox msg={error} />} // ⚠️ 0 به 0! حواست به
{status === "loading" ? <Spinner/> : <Content/>}
```

⚠️ تله کلاسیک &&: اگر سمت چپ عدد 0 شود، React خود 0 را رندر می‌کند! && {items.length} یا <List/> ← وقتی لیست خالی است، روی صفحه 0 می‌بینی! علاج: <List/> {items.length > 0 && <List/>} یا .ternary

۱.۳ – Props: فقط خواندنی و یک‌طرفه

- Props از پدر به فرزند جاری می‌شوند – فرزند هرگز نمی‌تواند prop پدر را عوض کند (one-way data flow)
- «تغییر props» در React ضدالگو است – تغییر باید با state و از صاحبش باشد
- ویژه: هرچی بین تگ باز و بسته بنویسی `children` = prop

```
function Card({ title, children }: { title: string; children: React.ReactNode }) {
  return (
    <div className="card">
      <h3>{title}</h3>
      {children} /* ای که والد داخلش گذاشت JSX هر */
    </div>
  );
}
```

۱.۴ – لیست‌ها و Key: پرتکرارترین سوال مصاحبه ★

```
const todos = [
  { id: 10, text: "خرید" },
  { id: 22, text: "مطالعه" },
];

<ul>
  {todos.map((t) => (
    <li key={t.id}>{t.text}</li>    { /* شناسه پایدار = key */ }
  ))}
</ul>
```

key برای چیست؟ React برای اینکه بداند کدام آیتم اضافه/حذف/جابجا شده، بین رندرها آیتم‌ها را با key تطبیق می‌دهد. key درست = حفظ state و DOM درست.

key	داوری
id پایدار از داده	✅ استاندارد
index آرایه	⚠️ فقط اگر: لیست ثابت است، هرگز reorder/sort نمی‌شود، وسطش insert نمی‌شود
Math.random()	❌ هر رندر عوض می‌شود – فاجعه state و performance

🔑 مثال کلاسیک خرابی: لیست با `key={index}` که آیتم اولش حذف می‌شود → همه key ها شیفت می‌شوند → React فکر می‌کند محتوای هر ردیف عوض شده → state اینپوت‌ها به ردیف‌های اشتباه می‌چسبد!

۱.۵ – رندر: چی اتفاق می‌افتد؟

چرخه هر رندر: Trigger (setState / اولین mount) → Render (صدا زدن کامپوننت‌ها) → Commit (به‌روزرسانی DOM)

- رندر یعنی «صدا زدن تابع کامپوننت» – نباید عوارض جانبی داشته باشد
- React در StrictMode در dev هر کامپوننت را دو بار صدا می‌زند تا ناخالصی‌ها را لو بدهد – عادی است، باگ نیست!

✅ جمع‌بندی فصل

- کامپوننت = تابع خالص با حرف بزرگ که JSX برمی‌گرداند
- `{}` برای JS؛ `&&` با `•` تله دارد؛ یک root
- props فقط خواندنی و یک‌طرفه؛ children = محتوای داخل تگ
- key = شناسه پایدار؛ index فقط برای لیست‌های کاملاً ثابت

- رندر = صدا زدن تابع؛ StrictMode دو بار صدا می‌زند عمداً

تمرین فصل ۱

۱. این کامپوننت سه ایراد دارد — پیدا کن:

```
function UserCard(user) {  
  user.name = user.name.toUpperCase();  
  return <div className=user-card>{user.name}</div>;  
}
```

۲. `{count && <Badge count={count} />}` چه زمانی خروجی عجیب می‌دهد؟ درستش کن.

۳. لیست کاربران قابل sort است — آیا `key={index}` مجاز است؟ چرا؟

۴. StrictMode چرا در dev کامپوننت‌ها را دو بار رندر می‌کند و در production چرا نه؟

▼ جواب‌ها

۱. (۱) props را mutate می‌کند (ناخالص) — باید `user.name.toUpperCase()` را در متغیر جدید؛ (۲) `user` به

صورت prop نام‌گذاری شده نیامده — باید `{ user } : { user: User }`؛ (۳) `className=user-card` بدون

کوئیشن/آکولاد — `className="user-card"`.

۲. وقتی `count = 0` است — JS مقدار ۰ را برمی‌گرداند و React عدد ۰ را رندر می‌کند. درست: `{count > 0 && <Badge count={count} />}`

`<Badge count={count} />}`

۳. نه — چون sort ترتیب را عوض می‌کند و key های index با آیتم‌ها تطبیق داده نمی‌شوند؛ state و DOM به هم

می‌ریزند. از `user.id` استفاده کن.

۴. برای کشف side-effect ها (ناخالصی)؛ چون در production این چک هزینه دارد و فرض بر این است که کد

ناخالصی ندارد.

→ **فصل بعد:** مدیریت State — snapshot ها، آپدیت immutable و الگوهای سازمان‌دهی.

فصل ۲ – مدیریت State (Managing State)

منبع رسمی: react.dev/learn/managing-state + Reference: `useState`، `useReducer`، `useContext`

هدف: state به عنوان snapshot، آپدیت `immutable`، `batching` و انتخاب درست بین `useState` / `useReducer` / `Context`

۲.۱ – State یک snapshot است (پرتکرارترین سوءتفاهم)

```
function handleClick() {  
  setCount(count + 1);  
  setCount(count + 1);  
  setCount(count + 1);  
  // مقدار نمایش داده شده: +1 فقط! چرا؟  
}
```

`count` یک متغیر عادی در این رندر است – در هر سه خط، همان مقدار رندر فعلی خوانده می‌شود. `setCount` = «درخواست رندر با مقدار جدید»، نه تغییر فوری متغیر.

علاج: آپدیت تابعی – وقتی به state قبلی وابسته‌ای:

```
function handleClick() {  
  setCount((c) => c + 1);  
  setCount((c) => c + 1);  
  setCount((c) => c + 1);  
}
```

سه بار = +3 ✓

قاعده طلایی: اگر state جدید از state قبلی محاسبه می‌شود، همیشه فرم تابعی بزن.

۲.۲ – Batching: چند `setState` در یک event

React 18 همه `setState` های یک event handler را در یک رندر جمع می‌کند (batching) – حتی داخل `setTimeout` و `promise` ها. پس نگران «سه `setState` = سه رندر» نباش.

۲.۳ – Immutable Update: فراموش نکردنی‌ها ★

state اشیاء و آرایه‌ها را جایگزین کن، نه تغییر بده – چون React با مرجع (reference) تغییر را تشخیص می‌دهد:

```
// ❌ mutation - React متوجه تغییر نمی‌شود
user.name = "Ali";
setUser(user);
cart.push(item);
setCart(cart);

// ✅ immutable - آرایه جدید
setUser({ ...user, name: "Ali" });
setCart([...cart, item]);

// زنجیره‌ای spread آپدیت تودرتو با:
setUser({
  ...user,
  address: { ...user.address, city: "تهران" },
});

// حذف و آپدیت در آرایه:
setTodos(todos.filter((t) => t.id !== id));
setTodos(todos.map((t) => (t.id === id ? { ...t, done: true } : t)));
```

💡 برای ساختارهای عمیق، Immer یا مدیریت state در سرور (TanStack Query) این درد را می‌گیرد – ولی اصل را باید بلد باشی.

۲.۴ – Lifting State Up: state بالایی مشترک‌ترین جد

دو کامپوننت خواهر به داده مشترک نیاز دارند؟ state را به نزدیک‌ترین پدر مشترک ببر و با props پایین بده:

```
function SearchPage() {
  const [query, setQuery] = useState(""); // اینجا بالای هر دو
  return (
    <
      <SearchBox value={query} onChange={setQuery} />
      <ResultList query={query} />
    </>
  );
}
```

سرچ‌باکس «controlled» است: مقدارش از props می‌آید (single source of truth).

۲.۵ – useReducer: وقتی منطق پیچیده می‌شود

وقتی چند فیلد state با هم مرتبط‌اند و transition های مشخصی دارند، reducer منطق را متمرکز می‌کند:

```

type State = { items: CartItem[]; total: number };
type Action =
  | { type: "add"; item: CartItem }
  | { type: "remove"; id: number }
  | { type: "clear" };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "add":
      return {
        items: [...state.items, action.item],
        total: state.total + action.item.price,
      };
    case "remove": {
      const item = state.items.find((i) => i.id === action.id!);
      return {
        items: state.items.filter((i) => i.id !== action.id),
        total: state.total - item.price,
      };
    }
    case "clear":
      return { items: [], total: 0 };
  }
}

const [state, dispatch] = useReducer(reducer, { items: [], total: 0 });
dispatch({ type: "add", item: newOne });

```

useReducer	useState
چند state مرتبط / منطق پیچیده	state مستقل و ساده
همه transition ها در یک تابع قابل تست	آپدیت‌ها پراکنده در کامپوننت

۲.۶ Context – رد کردن props از وسط درخت

مشکل «prop drilling» – داده باید از ۵ لایه رد شود. Context = پخش سراسری:

```

const ThemeContext = createContext<"light" | "dark">("light");

// در بالای درخت:
<ThemeContext.Provider value={theme}>
  <App />
</ThemeContext.Provider>

// در هر عمقی:
const theme = useContext(ThemeContext);

```

⚠️ دو هشدار داکيومنت: 1. Context برای داده‌های کم‌تغییر (تم، locale، کاربر جاری) است – نه state پرتغییر (هر تغییر = رندر همه مصرف‌کننده‌ها) 2. قبل از Context، اول lifting را امتحان کن؛ بسیاری از دریلینگ‌ها با composition (children) حل می‌شوند

۲.۷ – دو قانون دیفرنسینگ state (از داکيومنت)

1. State هایی که همیشه با هم تغییر می‌کنند را یکی کن – دو state `firstName/lastName` که همیشه با هم ست می‌شوند → یکی `fullName`
2. چیزهایی که از state دیگر محاسبه می‌شوند را state نکن – `filtered = items.filter(...)` محاسبه کن، state نگیر (وگرنه دوتا منبع حقیقت!)

✓ جمع‌بندی فصل

- state = snapshot رندر فعلی؛ آپدیت وابسته → فرم تابعی `(c) ≥ ...`
- batching: چند `setState` = یک رندر
- spread: immutable/film-map؛ هرگز mutation
- lifting state up؛ controlled components؛ composition قبل از Context
- `useReducer` برای ماشین‌حالت‌های پیچیده
- state محاسبه‌شدنی = محاسبه، نه state

📝 تمرین فصل ۲

1. خروجی چیست و چرا؟

```
const [age, setAge] = useState(20);
<button onClick={() => { setAge(age + 1); setAge(age + 1); }}></button>
```

2. کد سید خرید زیر باگ state دارد – درستش کن:

```
function addToCart(item: Item) {
  cart.items.push(item);
  setCart(cart);
}
```

3. یک فرم با سه state جدا (`name` , `email` , `message`) که همیشه با هم `reset` می‌شوند – پیشنهاد داکيومنت چیست؟

4. چه زمانی `useReducer` به جای چند `useState`؟ یک مثال واقعی از پروژه‌هایت بنویس.

▼ جواب‌ها

1. فقط $+1$ – هر دو فراخوانی از snapshot همان رندر می‌خوانند. درست: `setAge(a => a + 1)` دوبار.
2. mutation آرایه و پاس دادن همان مرجع $\rightarrow$ React rerender نمی‌کند. درست: `setCart({ ...cart, items: [...cart.items, item] })`
3. یکی‌کردن: یک آبجکت `form` با یک `setForm` + آپدیت `setForm({...form, name: v})` (immutable) یا یک `emptyForm` state برای `.reset`
4. مثال: state سبد خرید با `transition` – `add/remove/clear/qty-change` های مشخص، منطق متمرکز و قابل تست.

➔ فصل بعد: `useRef`، `useEffect` – `Escape Hatches` و جادوهای پشت `door` شماره ۳.

فصل ۳ – useRef، useEffect و Custom Hooks

📖 منبع رسمی: react.dev/learn/escape-hatches + Reference: useRef، useEffect، useMemo،

useCallback 🎯 هدف: دم به door سه – خروج از جریان رندر: دسترسی به DOM، همگام‌سازی با دنیای بیرون، و یادگیری اینکه useEffect «چرخه عمر» نیست بلکه همگام‌ساز است.

۳.۱ – useRef: جعبه‌ای که رندر نمی‌شود

ref = یک ظرف mutable که بین رندها حفظ می‌شود ولی تغییرش رندر ایجاد نمی‌کند:

```
const inputRef = useRef<HTMLInputElement>(null);  
  
<input ref={inputRef} />  
<button onClick={() => inputRef.current?.focus()}>فوکوس</button>
```

دو کاربرد اصلی: 1. DOM – فوکوس، scrollToView، اندازه‌گیری 2. مقدار بی‌رندر – timer id، شمارنده‌ی بین رندها، previous value

```
const renderCount = useRef(0);  
renderCount.current++; // تغییرش رندر نمی‌سازد - فقط ذخیره
```

ref	state	
❌ نه	✅ بله	تغییر → رندر؟
❌ نباید در رندر خوانده شود	بله (snapshot)	خواندن در رندر؟
مقدار فنی/DOM	داده نمایشی	مناسب

۳.۲ – useEffect: همگام‌سازی با دنیای بیرون

مهم‌ترین بازتعریف داکيومنت: Effects برای همگام‌سازی با سیستم‌های بیرونی‌اند (سرور، DOM API، subscription، غیر ری‌اکتی)، نه برای منطق عادی:

```
useEffect(() => {
  const conn = createConnection(roomId); // با سیستم بیرون همگام شو
  conn.connect();
  return () => conn.disconnect(); // cleanup: قبلی را قطع کن
}, [roomId]); // عوض شد roomId وقتی
```

چرخه ذهنی درست: تغییر prop/state → رندر → DOM آپدیت → اگر وابستگی عوض شده، cleanup قبلی + effect جدید

وابستگی‌ها (dependencies)

- آرایه خالی [] = فقط بعد از mount
- [roomId] = با تغییر roomId
- همه reactive value های داخل effect باید در آرایه باشند (دکمه‌ی اضافه کردن خودکار lint) – دروغ گفتن به وابستگی‌ها = باگ‌های عجیب

تله‌های معروف ★

(۱) آبجکت/تابع به عنوان وابستگی:

```
useEffect(() => { ... }, [options]); // آبجکت جدید در هر رندر = بی‌نهایت اجرا options ✗
```

علاج: وابستگی‌های اتمی ([options.host, options.port]) یا useMemo.

(۲) race condition در fetch:

```
useEffect(() => {
  let ignore = false;
  fetch(`/api?q=${query}`)
    .then((r) => r.json())
    .then((d) => { if (!ignore) setResult(d); });
  return () => { ignore = true; }; // پاسخ دیر هنگام قبلی نادیده گرفته می‌شود
}, [query]);
```

(۳) Effects که لازم نیستند – از داکيومنت «You Might Not Need an Effect» - state محاسبه‌شدنی → در رندر محاسبه کن - واکنش به تغییر prop → در رندر (بدون state اضافی) - event handling → در event handler، نه effect fetch - داده → در اپ‌های جدی با فریم‌ورک (Next.js!) یا TanStack Query – نه useEffect خام

۳.۳ Custom Hooks: منطق مشترک با قواعد React

تابعی که خودش از hook ها استفاده می‌کند – نامش با use شروع می‌شود:

```
function useDebounceValue<T>(value: T, delay = 300): T {
  const [debounced, setDebounce] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounce(value), delay);
    return () => clearTimeout(id); // هر تغییر جدید، تایمر قبلی را لغو می‌کند
  }, [value, delay]);

  return debounced;
}

// مصرف:
const debouncedQuery = useDebounceValue(query);
useEffect(() => { if (debouncedQuery) search(debouncedQuery); }, [debouncedQuery]);
```

Custom hook = **اشتراک منطق** (نه اشتراک state – هر تماس، state مستقل خودش را دارد).

۳.۴ – useMemo و useCallback و Performance Hooks

```
const filtered = useMemo(
  () => hugeList.filter((i) => i.active),
  [hugeList]
);

const handleSave = useCallback((id: number) => {
  save(id);
}, []);
```

hook	چه چیزی را کش می‌کند	مصرفش وقتی؟
useMemo	نتیجه محاسبه سنگین یا آبجکت پایدار برای وابستگی دیگری	محاسبه واقعاً سنگین یا مرجع ثابت لازم است
useCallback	خود تابع	پاس به کامپوننت memo شده یا وابستگی effect

⚠️ **داکیومنت React 19** لحنش این است: به صورت پیش‌فرض استفاده نکن – compiler آینده (React Compiler) خودش memo می‌کند؛ فقط جاهای واقعاً اندازه‌گیری شده اضافه کن. (مثل بهینه‌سازی کورکورانه DB – اول بچسب، بعد بهینه کن!)

۳.۵ – useSyncExternalStore (یک خط برای اشتراک store خارجی)

وقتی به store خارجی (نه state ری‌اکتی) وصل می‌شوی:

```
const online = useSyncExternalStore(subscribe, () => navigator.onLine);
```

عملاً پشت TanStack Query/Zustand است – شناختنش برای درک زیرساخت کافی است.

جمع‌بندی فصل ✓

- ref = جعبه‌ی بی‌رندر (DOM + مقادیر فنی)؛ state را جایگزین نکن
- useEffect = همگام‌سازی بیرون؛ dependency ها صادقانه؛ cleanup همیشه
- fetch در effect = race با ignore – یا ابزار درست‌تر
- Custom hook = اشتراک منطق؛ naming با use
- useMemo/useCallback اندازه‌گیری‌شده، نه همه‌جا

تمرین فصل ۳ 📄

1. این کد چه باگی دارد؟

```
const [timer, setTimer] = useState<number>();
useEffect(() => {
  const id = setInterval(tick, 1000);
  setTimer(id);
}, []);
```

2. جستجوی زنده با fetch در useEffect نوشتی؛ کاربر سریع تایپ می‌کند – دو مشکل و علاجه‌ها؟
3. کدام را با ref و کدام را با state نگه می‌داری؟ (الف) مقدار toggle تم (ب) id تایمر (ج) مقدار input جستجو (د) آخرین scrollY
4. چرا `useEffect(() => {...}, [options])` با آبجکت options مشکل دارد و سه علاجه‌اش؟

▼ جواب‌ها

1. تایمر هرگز clear نمی‌شود (لغو subscribe) – cleanup بنویس: `return () => clearInterval(id);` و لازم نیست تایمر state باشد – ref کافی است.
2. race condition (۱) – پاسخ دیرتر ممکن است قدیمی‌تر باشد → flag ignore یا AbortController؛ (۲) درخواست برای هر کلید! → debounce با custom hook.
3. state: (الف) و (ج)؛ ref: (ب) و (د).
4. آبجکت در هر رندر مرجع جدید دارد → effect بی‌نهایت اجرا می‌شود. علاجه‌ها: وابستگی اتمی به فیلدهایش، useMemo برای پایدارسازی، یا تعریف آبجکت بیرون کامپوننت.

➔ فصل بعد: React 19 و پترن‌های مدرن – use، useActionState، useOptimistic.

فصل ۴ – React 19 و پترن‌های مدرن

منبع رسمی: react.dev/reference/react-use, `useActionState`, `useFormStatus`, `useOptimistic`, `useTransition`

هدف: `useTransition` (🎯) هوک‌های نسل جدید React 19 که Next.js 16 روی آن‌ها سوار است – مخصوصاً سه‌گانه‌ی فرم‌ها که در فصل ۸ دوباره به کارت می‌آید.

۴.۱ – `useTransition`: رندر غیرفوری (Non-blocking)

آپدیت «کند» (فیلتر لیست بزرگ) را کم‌اولویت کن تا UI فوری (اینپوت) گیر نکند:

```
const [isPending, startTransition] = useTransition();

function onChange(e: React.ChangeEvent<HTMLInputElement>) {
  setQuery(e.target.value); // فوری: اینپوت روان می‌ماند
  startTransition(() => {
    setFiltered(hugeFilter(e.target.value)); // کم‌اولویت: قابل قطع
  });
}

{isPending && <SpinnerMini />}
```

`useDeferredValue(value)` معادل ساده‌ترش است – «نسخه عقب‌مانده از value» را بده و React let اول UI فوری را رندر کند.

۴.۲ – `use`: خواندن `Promise` و `Context` با شرط

هوک جدید که برخلاف بقیه می‌تواند داخل شرط و حلقه صدا زده شود:

```
import { use } from "react";

function Comments({ commentsPromise }: { commentsPromise: Promise<Comment[]> }) {
  const comments = use(commentsPromise); // Suspense را فعال می‌کند تا promise حل شود
  return comments.map((c) => <Comment key={c.id} {...c} />);
}

// برای حالت بارگذاری <Suspense> + را می‌سازد و می‌فرستد promise: والد
<Comments commentsPromise={fetchComments()} />
```

همچنین جایگزین `useContext` است: `const theme = use(ThemeContext);`

این همان الگویی است که Server Component ها در Next.js با `prop` ی `promise` (مثل فصل ۹ دوره وردپرس Headless) استفاده می‌کنند. 💡

۴.۳ – useActionState: فرم + state نتیجه اکشن

جایگزین مدرن چرخه‌ی «loading → submit → نتیجه/ارور»:

```
const [state, formAction, isPending] = useActionState(
  async (prevState, formData) => {
    const name = String(formData.get("name") ?? "");
    const error = name.trim() ? null : "نام الزامی است";
    if (error) return { error };
    await saveName(name);
    return { ok: true };
  },
  { error: null }
);

<form action={formAction}>
  <input name="name" />
  {state.error && <p className="text-red-600">{state.error}</p>}
  <button disabled={isPending}>{isPending ? "...": "ذخیره"}</button>
</form>
```

پیش‌فرض در Next.js: وقتی action یک Server Action باشد (فصل ۸)، همین الگو سراسری است.

۴.۴ – useFormStatus: حالت فرم از داخل فرزند

دکمه‌ی submit معمولاً در کامپوننت جداست و به state فرم دسترسی ندارد – این هوک همان فرم پدر را می‌خواند:

```
function SubmitButton() {
  const { pending } = useFormStatus(); // ⚠ باید داخل <form> باشد
  return <button disabled={pending}>{pending ? "ثبت": "... در حال ارسال"}</button>;
}

<form action={formAction}>
  <input name="name" />
  <SubmitButton />
</form>
```

۴.۵ – useOptimistic: به‌روزرسانی خوش‌بینانه

قبل از جواب سرور، UI را آپدیت کن – اگر شکست خورد برمی‌گردد:

```
const [optimisticLikes, addOptimistic] = useOptimistic(likes, (cur, delta: number) => cur + delta);

async function handleLike() {
  addOptimistic(1); // دیده می‌شود UI فوری در
  await likePost(postId); // درخواست واقعی
  startTransition(() => { refreshLikes(); }); // همگام‌سازی نهایی با سرور
}
```

تجربه‌ی UX اینستاگرامی: لایک فوری، همگام‌سازی پشت صحنه.

۴.۶ – ref به عنوان prop + ref cleanup (React 19)

از ۱۹ به بعد دیگر `forwardRef` لازم نیست – `ref` یک prop معمولی است:

```
function MyInput({ ref, ...props }: { ref?: React.Ref<HTMLInputElement> }) {  
  return <input ref={ref} {...props} />;  
}  
// جدید در ۱۹: callback برگرداندن تابع از cleanup ref و  
<input ref={node} => { register(node); return () => unregister(node); } />
```

۴.۷ – چیزهای دیگر ۱۹ در یک نگاه

قابلیت	چیست
<code>use server</code> / <code>use client</code>	directive ها (پایه Server Actions فصل ۸)
Server Components پایدار	در Next.js از قبل استفاده می‌کردی
بهبود hydration errors	پیام ارور با diff واقعی DOM
<code><Context></code> به جای Provider	<code><ThemeContext value={theme}></code>
Document metadata	<code><title></code> / <code><meta></code> در کامپوننت‌ها (در Next.js با Metadata API بهتر است – فصل ۱۰)

✓ جمع‌بندی فصل

- `useTransition` / `useDeferredValue` = آپدیت کند، UI روان
- `use` = خواندن promise/context در هر جایی (Suspense-friendly)
- سه‌گانه فرم: `useActionState` (نتیجه)، `useFormStatus` (pending از فرزند)، `useOptimistic` (UX فوری)
- `ref` به عنوان prop معمولی؛ `forwardRef` goodbye
- `memo` کردن: اندازه‌گیری‌شده – React Compiler در راه است

📝 تمرین فصل ۴

1. تفاوت `useDeferredValue` و `useTransition` را با یک مثال بگو – کی هرکدام؟
2. چرا `useActionState` سه عضو برمی‌گرداند و نقش هرکدام؟
3. سناریو: در چت، پیام کاربر بلافاصله در لیست بیاید و ارسال در پس‌زمینه انجام شود – کدام هوک؟ اسکلت کد بنویس.
4. `useFormStatus` را در کامپوننتی خارج از `<form>` صدا زدی – چه می‌بینی و چرا؟

1. `useTransition`: آپدیت‌های «خودم اجرا می‌کنم» را کم‌اولویت می‌کند (نوشتن منطق)؛ `useDeferredValue`: مقدار را «دیرتر» می‌دهد وقتی نمی‌خواهی منطق را دستکاری کنی (فقط `consumption` کند است). برای فیلتر حین تایپ هر دو؛ برای `prop` دریافتی فقط `deferred`.
2. `state` (خروجی آخرین اکشن)، `formAction` (برای `action` فرم)، `isPending` (در حال اجرا بودن اکشن).
3. `useOptimistic`:

```
const [optimisticMessages, addOptimistic] = useOptimistic(messages,
  (cur, msg: Message) => [...cur, { ...msg, pending: true }]);
async function send(text: string) {
  const msg = { id: crypto.randomUUID(), text };
  addOptimistic(msg);
  await postMessage(msg);
  startTransition(() => { refresh(); });
}
```

4. همیشه `pending: false` – چون `useFormStatus` به نزدیک‌ترین `<form>` والدش (در همان درخت رندر) وصل است؛ بیرون فرم، کانتکستی نمی‌بیند.

فصل ۵ – Next.js: ساختار پروژه و Routing

منبع رسمی: nextjs.org/docs/app – Project Structure, Layouts and Pages, Linking and Navigating

+ App Router – layout/page/loading/error/not-found/route هدف: قرارداد فایل محور
ناوبری Link – با نکات پشت صحنه ای که در مصاحبه می پرسند.

۵.۱ – Routing = پوشه بندی

App Router یعنی «پوشه = مسیر URL». قراردادهای فایل ویژه (special files):

فایل	نقش
page.tsx	محتوای عمومی مسیر – بدون آن مسیر قابل دسترسی نیست!
layout.tsx	قاب مشترک مسیر و فرزندانش (nav, sidebar) – state حفظ می شود
loading.tsx	fallback حین رندر = Suspense خودکار
error.tsx	مرز خطا (باید "use client" باشد)
not-found.tsx	صفحه 404 برای notFound() این بخش
route.ts	Route Handler – API endpoint (این پوشه page نمی گیرد)
template.tsx	مثل layout ولی در هر ناوبری remount می شود (انیمیشن ها)
default.tsx	fallback برای slot های ناتمام در parallel routes

ساختار نمونه:

```
app/
├── layout.tsx      ← (اینجاست html/body) ریشه
├── page.tsx        ← /
├── blog/
│   ├── page.tsx   ← /blog
│   └── [slug]/
│       └── page.tsx ← /blog/:slug
├── dashboard/
│   ├── layout.tsx ← قاب داشبورد
│   ├── settings/page.tsx
│   └── loading.tsx ← فقط زیر داشبورد!
├── api/
│   └── products/route.ts ← /api/products
```

۵.۲ – Layout ها: نکاتی که وسط کار یادت می‌رود

```
// app/layout.tsx – فقط layout می‌تواند ریشه <html> و <body> باشد
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="fa" dir="rtl">
      <body>{children}</body>
    </html>
  );
}
```

- layout ها دوباره رندر نمی‌شوند هنگام نوبری داخل‌شان – state و scroll حفظ است (برخلاف template)
- layout می‌تواند async باشد و دیتا بگیرد (Server Component است)
- فرزندان را نمی‌بیند، فقط `children` می‌گیرد – به مسیر فعلی دسترسی ندارد (اگر لازم شد: کامپوننت Client با `usePathname` یا `middleware`)

Route Groups و پوشه‌های بدون URL

- `(marketing)/about/page.tsx` → مسیر `/about` – پرانتز یعنی «فقط برای سازمان‌دهی، در URL نمی‌آید» (دو layout موازی بدون تغییر مسیر)
- `_components/` → آندرلین اول = «private» – از routing مستثنی

Dynamic segments

```
// app/blog/[slug]/page.tsx
export default async function Page({ params }: { params: Promise<{ slug: string }> }) {
  const { slug } = await params; // ⚠️ در Next 15+ params یک Promise است!
}

// [...slug] catch-all – /a/b/c را هم می‌گیرد
// [?. slug] optional catch-all – / را هم می‌گیرد
```

۵.۳ – نوبری: Link (نه !)

```
import Link from "next/link";

<Link href="/blog/42" prefetch>۴۲ پست</Link>
<Link href={` /blog?tag=${tag}`} scroll={false} replace>...</Link>
```

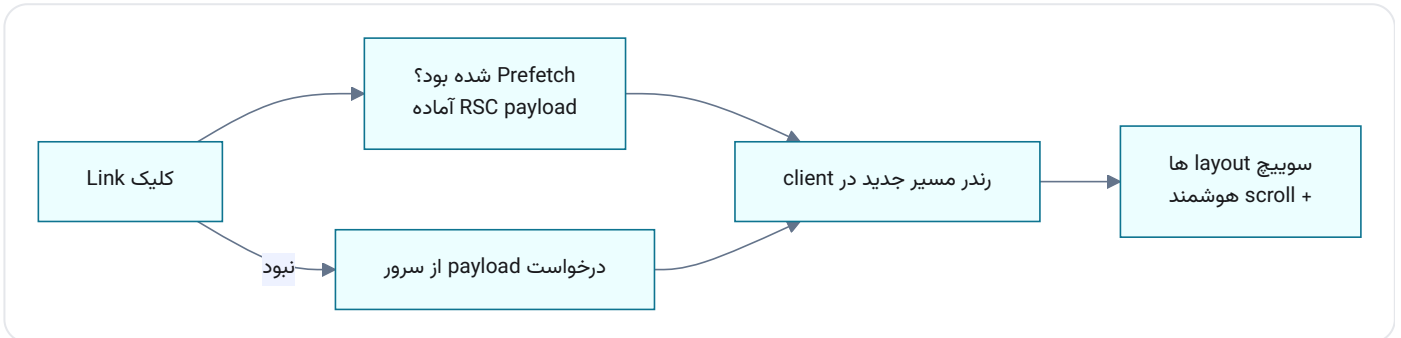
چرا Link به جای **Client-side navigation**: 1. `<a>` – بدون reload کامل صفحه 2. **Prefetch** خودکار – وقتی لینک وارد viewport می‌شود، مسیر پیش‌بارگذاری می‌شود (در production) → نوبری حس فوری 3. نگاه داشتن state layout ها و scroll هوشمند

پرچم‌ها: `prefetch={false}` برای لینک‌های خروجی/ناکثریت؛ `replace` به جای `push` در history؛ `scroll={false}` برای حفظ اسکرول.

نکته: **useRouter** برای ناوبری برنامه‌ای:

```
"use client";
import { useRouter, usePathname, useSearchParams } from "next/navigation";
const router = useRouter();
router.push("/dashboard"); // router.replace / back / refresh
const pathname = usePathname(); // مسیر فعلی (برای هایلایت منو)
const searchParams = useSearchParams(); // ⚠️ یک hook است - کامپوننت client یا Suspense boundary دارد
```

۵.۴ – ناوبری چطور کار می‌کند (پشت صحنه)



ناوبری در App Router یعنی جابجایی **RSC payload** (نه HTML کامل) – به همین دلیل layout های مشترک لمس نمی‌شوند.

✓ جمع‌بندی فصل

- پوشه = مسیر؛ فایل‌های ویژه نقش‌ها را می‌دهند؛ بدون **page.tsx** مسیری نیست
- **layout**: قاب با state پایدار؛ فقط ریشه **html/body** دارد؛ **async** مجاز
- **(group)** سازمان‌دهی بدون URL؛ **_folder** خصوصی؛ **[param]** و **catch-all**
- **params/searchParams** یک **Promise** اند – **await** کن (Next 15+)
- **Link** = ناوبری **prefetch** + **client**؛ **router.push** برای برنامه‌ای

📝 تمرین فصل ۵

1. ساختار مسیرهای زیر را طراحی کن (پوشه‌ها): **/dashboard/settings/profile** ، **/dashboard/overview** با یک قاب مشترک داشبورد و **loading** مخصوص فقط برای **settings**.
2. چرا این ارور می‌گیری؟ **app/about/layout.tsx** شامل **<html>** است – چه بلایی سر سایت می‌آید؟
3. **app/blog/page.tsx** و **app/blog/route.ts** همزمان داشته باشی چه می‌شود؟
4. چرا منوی فعال‌ساز (active link) با **usePathname** ساخته می‌شود و کامپوننتش **"use client"** دارد – **layout** چرا به تنهایی نمی‌تواند؟

5. فرق `router.push` و `router.replace` در یک سناریوی واقعی (لاگین!) بگو.

▼ جواب‌ها

```
app/dashboard/layout.tsx      ← قاب مشترک
app/dashboard/overview/page.tsx
app/dashboard/settings/
├─ loading.tsx                ← فقط شاخه settings
└─ profile/page.tsx
```

2. فقط `layout` ریشه مجاز به `html/body` است؛ تودرتویی، دو تگ `html` ایجاد می‌کند (یا ارور) – `layout` داخلی فقط قاب محتوا است.

3. تعارض! یک مسیر نمی‌تواند هم `page` و هم `route` داشته باشد – `build` ارور می‌دهد. API را در `app/api/blog/route.ts` بگذار.

4. `layout` سرور است و به مسیر فعلی دسترسی ندارد (`children` می‌گیرد)؛ `pathname` فقط سمت کلاینت با `usePathname` در دسترس است.

5. بعد از لاگین، به `history` صفحه لاگین نباید برگشت (`back` → خروج!) – `router.replace("/dashboard")`.

→ فصل بعد: ★ مهم‌ترین فصل – Server vs Client Components.

فصل ۶ – Server vs Client Components (قلب App Router)

منبع رسمی: nextjs.org/docs/app/building-your-application/rendering – Server and Client Components, Composition Patterns

هدف: مهم‌ترین فصل برای مصاحبه و مهم‌ترین منبع باگ‌های روزمره – مدل ذهنی کامل RSC: کی سرور، کی کلاینت، مرزها و ترکیب‌ها.

۶.۱ – مدل ذهنی: پیش‌فرض سرور است

در App Router همه‌چیز به صورت پیش‌فرض **Server Component** است – نه چون «سرور رندر می‌کند»، چون کد آن فقط روی سرور اجرا و خروجی‌اش (RSC payload) به مرورگر می‌رود:

Server Component (پیش‌فرض)	Client Component ("use client")	
فقط سرور (یک بار در build/request)	سرور (SSR اولیه) + مرورگر (hydrate/interactivity)	اجرا
❌ صفر	✅ کدش می‌رود	JS فرستاده به کلاینت
✅ مستقیم	❌ هرگز	دیتابیس/فایل/secret
❌ ندارد	✅ دارد	useState/useEffect/hooks مرورگری
❌ کار نمی‌کند	✅	onClick/onSubmit
✅ مستقیم	❌ (با useEffect/use یا پدر سرور)	async/await در خودش

🔑 جمله قاب‌شدنی: «Client Component یعنی interactive است، نه اینکه فقط در کلاینت اجرا شود.» – اولین رندرش هم روی سرور HTML می‌شود.

۶.۲ – مرز: "use client" یعنی «در زیر من کلاینتی»

directive روی اول فایل، تمام import های آن فایل را کلاینت می‌کند – مرز، یک‌طرفه از آن نقطه به پایین است:

```

app/page.tsx (Server) ✓
└─ FilterPanel.tsx ("use client") 📦 کلاینتی
    └─ FilterList.tsx 📦 کلاینتی (بدون directive)
    └─ DataService.ts ✗ import کار نمی‌کند RSC آن به

```

پیامدها: - یک کامپوننت کلاینتی نمی‌تواند async باشد و مستقیم دیتابیس بخواند - اما می‌تواند زیر خودش، کامپوننت سروری را به عنوان children دریافت کند! (بخش ۶.۴)

۶.۳ - پاس دادن داده از سرور به کلاینت: Serialization ★

مرز سرور → کلاینت یک «مرز سریال‌سازی» است - فقط داده‌های serializable رد می‌شوند:

رد می‌شود ✗	رد می‌شود ✓
توابع	آبجکت/آرایه/رشته/عدد/بولین/null
کلاس‌های با متد/behavior	Date, Map, Set, Promise (RSC)
import شدن مستقیم کامپوننت سروری در فایل کلاینتی	React Element (JSX)

```

// ✓ Server Component:
<ProductCard product={product} /> // ✓ آبجکت ساده
<Chart getData={loadData} /> // ✗ تابع - cross نمی‌شود!

```

۶.۴ Composition: پترن‌های طلایی ★

الگوی «کلاینت لُزُم» - تا جای ممکن پایین درخت کلاینت کن:

```

// ✓ Server (page.tsx):
<ClientInteractivePiece data={serializable} />

```

و الگوی children/passthrough - وقتی کامپوننت کلاینتی می‌خواهد «چیزی سروری» را نشان بدهد:

```

// ✗ import راند <ServerOnlyStuff/> اشتباه رایج: کلاینت نمی‌تواند
"use client";
function Sidebar() {
  return <div>{ /* بگذارم ServerChart می‌خواهم اینجا */ }</div>;
}

// ✓ از پدر سروری children/props درست: کامپوننت‌های سروری به عنوان
// page.tsx (Server):
<Sidebar>
  <ServerChart />
</Sidebar>

// هنوز سروری است! فقط از داخل کلاینتی «عبور» داده شد ←
<Sidebar>
  <ServerChart />
</Sidebar>

// Sidebar.tsx ("use client"):
function Sidebar({ children }: { children: React.ReactNode }) {
  return <aside>{children}</aside>; // children می‌دهد
}

```

چرا کار می‌کند؟ چون ServerChart قبل از رسیدن به مرز روی سرور رندر شده و به صورت خروجی رندر شده داخل props عبور کرده.

Context و سرور

Provider ها (ThemeContext و...) چون state دارند کلاینتی‌اند – الگوی استاندارد: خود Provider را کلاینت کن ولی children را از سرور بگیر:

```
// theme-provider.tsx ("use client") - context فقط منطق
export function ThemeProvider({ children }) { ... }

// layout.tsx (Server):
<ThemeProvider>{children}</ThemeProvider>  // کل درخت سروری زیرش می‌ماند ✓
```

۶.۵ – کی کدام؟ (جدول تصمیم)

کامپوننت	باشد
داده می‌گیرد (API، db، فایل)	Server
secret / کلید API مصرف می‌کند	Server
وابستگی سنگین (دیتابیز، ادیتور)	Server (اگر تعاملی نیست) – باندل صفر!
state / effect / رویداد / browser API	Client
Context مصرف می‌کند	Client

قاعده عملی: از Server شروع کن؛ فقط جایی که interactivity لازم شد، یک جزیره کلاینت کوچک بساز – و آن جزیره را تا جای ممکن پایینِ درخت ببر.

۶.۶ – عوارض جانبی مرز (که فقط با تجربه می‌فهمی)

علامت	علت	علاج
<code>useState is not a function in Server Component</code>	hook در کامپوننت سروری	یا <code>"use client"</code> یا تقسیم کامپوننت
<code>Event handlers cannot be passed to Client Component props</code>	پاس دادن تابع به کلاینت از سرور	تابع را داخل کامپوننت کلاینی تعریف کن یا Server Action بده (فصل ۸)
یک کتابخانه کلاینتی کل صفحه را کلاینت کرد	directive در فایل بالادستی	مرز را پایین ببر – فایل ورودی کلاینی کوچک بساز
props با تابع: <code>Functions cannot be passed...</code>	serialization	Server Action (فصل ۸) یا داده به جای تابع

✓ جمع‌بندی فصل

- پیش‌فرض = Server؛ کدش به کلاینت نمی‌رود؛ db/secret فقط آنجا
- `"use client"` = مرز یک‌طرفه به پایین؛ کلاینت = SSR اولیه + interactivity
- مرز = serializable: داده ✓، تابع ✗ (به جز Server Actions)
- children/passthrough = عبور کامپوننت سروری از میان کلاینی
- Provider کلاینی با children سروری – الگوی استاندارد
- جزیره‌های کلاینی کوچک، پایین درخت

📄 تمرین فصل ۶

۱. کدام فایل‌ها کلاینتی‌اند؟ (فقط اسم بگو)

```
A.tsx (دیتابیس fetch دارد؟ نه – فقط useState، directive بدون)
B.tsx (را رندر می‌کند <C/>، در بالا "use client")
C.tsx (استاتیک div فقط، directive بدون)
D.tsx (directive دارد بدون onClick می‌شود؛ از A import)
```

۲. چرا این کد ارور می‌دهد و دو راه حل؟

```
// page.tsx (Server)
<DatePicker onChange={{(d) => saveDate(d)}} />
```

۳. کامپوننت کلاینی می‌خواهد داده دیتابیس را نمایش دهد – سه راه درست (بدون اتصال مستقیم DB از کلاینت).

4. سناریو: یک صفحه سروری + یک «ناوبری تب‌های محلی» (state دار) + محتوای هر تب سروری رندر می‌شود – طراحی کن.

▼ جواب‌ها

1. A سروری (بدون hook)؛ B کلاینی (directive)؛ C کلاینی (زیر مرز B – هرچند استاتیک است و می‌شد سروری بماند اگر از A فراخوانی می‌شد)؛ D کلاینی است به شرطی که داخل B مصرف شود (onClick در درخت کلاینی کار می‌کند). اگر D مستقیم از A صدا شود → ارور.
2. تابع از سرور به کلاینت پاس نمی‌شود (serialization). راه ۱: DatePicker کلاینی باشد و onChange را داخل خودش با Server Action صدا بزند؛ راه ۲: Server Action به عنوان prop (فصل ۸ – Server Action ها قابل پاس شدن از سرور به کلاینت‌اند!).
3. (۱) خود کامپوننت سروری بماند و داده را بگیرد، فقط بخش تعاملی‌اش کلاینی؛ (۲) fetch + Route Handler در کلاینت؛ (۳) Server Action و برگرداندن داده.
4. layout/page سروری → محتوای هر تب را به عنوان آرایه‌ای از ReactNode آماده کند → `<Tabs`
`serverContent={[{label, node}, ...]} />` کلاینی فقط state تب فعال را نگه دارد و node مربوطه را رندر کند – الگوی children/passthrough.

→ فصل بعد: Data Fetching – SSG/ISR/SSR و تایمینگ دقیق‌شان.

فصل ۷ – Data Fetching و استراتژی‌های رندر (SSG/ISR/SSR)

منبع رسمی: nextjs.org/docs/app/building-your-application/data-fetching – Fetching، Caching،

Revalidating 🔄 هدف: چهار استراتژی رندر و تایمینگ دقیق انتخاب‌ها + کش در Next 16 + fetch های موازی/زنجیره‌ای.

۷.۱ – چهار استراتژی در یک جدول

استراتژی	کی HTML ساخته می‌شود	مناسب	چطور فعال شود
SSG (Static)	در build، یک بار	محتوای کم‌تغییر (لندینگ، مستندات)	پیش‌فرض! هرچی می‌تواند static است
ISR	در build + بازتولید دوره‌ای	محتوای تغییرپذیر با تحمل تأخیر (بلاگ، فروشگاه)	<code>revalidate = 60</code>
SSR (Dynamic)	هر درخواست	داده شخصی/همیشه‌تازه (داشبورد، سشن)	توابع dynamic مثل <code>cookies()</code> یا <code>no-store</code>
CSR	در مرورگر	داده واقعاً کلاینتی/لحظه‌ای	کامپوننت + <code>client</code> <code>fetch/useEffect/SWR</code>

🔑 فلسفه static: Next 15/16 پیش‌فرض – سرور خودش تشخیص می‌دهد چه چیزی صفحه را dynamic می‌کند. کار تو «اجبار» نیست، «نشان دادن قصد» است.

۷.۲ – آنچه صفحه را Dynamic می‌کند

این‌ها در زمان رندر استفاده شوند، صفحه dynamic می‌شود:

```
import { cookies, headers } from "next/headers";

const c = (await cookies()).get("session"); // سشن = هر کاربر متفاوت
const ua = (await headers()).get("user-agent");
// و: searchParams در صفحه fetch با cache: "no-store"
```

و برعکس، برای «تازه نگه‌داشتن بدون SSR کامل»:

```
// کل صفحه بعد از ۶۰ ثانیه بازتولید:
export const revalidate = 60;

// بدون کش fetch فقط یک:
const res = await fetch(url, { cache: "no-store" }); // هر بار تازه fetch این

// یا بازاعتبارسنجی برچسب‌دار:
const res = await fetch(url, { next: { tags: ["products"] } });
revalidateTag("products"); // در Server Action بعد از mutation
```

💡 Next 16: سیستم کش با `use cache` و `cacheLife/cacheTag` مدرن شده – مدل ذهنی ثابت است: `cached by default`، قصد را اعلام کن؛ سینتکس‌ها در داکيومنت چک شود.

۷.۳ – کجا fetch کنیم؟ (در Server Component!)

```
// ✅ مستقیم در Server Component:
export default async function ProductsPage() {
  const res = await fetch("https://api.example.com/products", {
    next: { revalidate: 300 },
  });
  const products = await res.json();
  return <List items={products} />;
}
```

- بدون `useEffect`، بدون `state loading` – چون سرور صبر می‌کند و HTML آماده می‌فرستد
- `fetch` های با URL یکسان در یک رندر یک‌بار زده می‌شوند (Request Memoization)
- در Client Component: فقط با SWR/TanStack یا `useEffect` (+محدودیت‌های فصل ۳)

موازی در برابر زنجیره‌ای ⭐

```
// ❌ ۲۰۰ms زنجیره‌ای: ۱۰۰ms + ۳۰۰ms = ۴۰۰ms
const artist = await getArtist(username);
const albums = await getAlbums(artist.id);

// ✅ ۳۰۰ms موازی:
const [artist, albums] = await Promise.all([getArtist(username), getAlbums()]);
```

و اگر یکی برای رندر اولیه ضروری نیست، آن را پایین‌تر با `Suspense` استریم کن (فصل ۹):

```
// حل کن Suspense را پاس بده و در کامپوننت پایین‌تر با promise – await بدون
<Suspense fallback=<AlbumsSkeleton />>
  <Albums promise={getAlbums()} />
</Suspense>
```

۷.۴ – `generateStaticParams`: مسیرهای دینامیک static

برای `/blog/[slug]` اگر می‌خواهی صفحات در `build` ساخته شوند:

```
export async function generateStaticParams() {
  const posts = await getPosts(); // ها slug لیست
  return posts.map((p) => ({ slug: p.slug }));
}
// در build: یک صفحه slug برای هر HTML
// ساخته و کش می‌شوند request های جدید در slug (پیش‌فرض true): dynamicParams با
```

- ISR روی مسیرهای دینامیک: همان `export const revalidate = 3600` – لیست‌ها و صفحات هر ساعت تازه.

۷.۵ – searchParams: فیلترهای URL

```
export default async function ProductsPage({
  searchParams,
}: {
  searchParams: Promise<{ q?: string; page?: string }>;
}) {
  const { q, page = "1" } = await searchParams; // ⚠ Promise + رشته!
  const items = await searchProducts(q, Number(page));
}
```

- نکته‌ها: - ورود به searchParams صفحه را **dynamic** می‌کند (هر کوئری متفاوت = رندر متفاوت) - مقادیر همیشه string اند – تبدیل کن - تغییر searchParams از کلاینت: `{q=${q}}` `router.push(\`?q=${q}\`)` – سرور دوباره رندر می‌کند و URL برای اشتراک/سئو تمیز می‌ماند (الگوی فیلتر استاندارد)

۷.۶ – تصویر بزرگ: کدام صفحه چه باشد؟

صفحه	انتخاب
لندینگ	SSG خالص
بلاگ (لیست + پست)	ISR (<code>revalidate</code>) یا SSG + webhook
صفحه محصول	ISR
داشبورد کاربر	SSR (cookies) + استریم بخش‌های کند
جستجو/فیلتر	dynamic با searchParams
چت/تیکر زنده	SSR پویا + CSR/SSE برای داده لحظه‌ای

✓ جمع‌بندی فصل

- static پیش‌فرض؛ `revalidate` = ISR ؛ `cookies/headers/searchParams/no-store` = dynamic
- `fetch` در `Server Component`؛ `memoization` خودکار؛ `Promise.all` برای موازی

- generateStaticParams = مسیرهای دینامیک static؛ slug های جدید خودکار کش
- searchParams: Promise + string + dynamic
- fetch سنگین دوم؟ پایین تر ببرش و استریم کن

تمرین فصل ۷

1. برای هر صفحه بگو SSG/ISR/SSR/CSR و چرا: (الف) مستندات محصول (ب) سبد خرید کاربر (ج) لیست مقالات بلاگ (د) پروفایل عمومی کاربران با آواتار.
2. این صفحه چرا static رندر نمی شود؟

```
export default async function Page() {
  const data = await fetch(url, { cache: "no-store" }).then(r => r.json());
  return <Dashboard data={data} />;
}
```

3. کد فچ زنجیره ای سه مرحله ای را موازی کن؛ کدام مرحله واقعاً وابسته است؟
4. چرا در Server Component از useEffect برای fetch استفاده نمی کنیم؟ دو دلیل.
5. فیلتر قیمت در (URL ?min=100) – پیاده سازی سرور + کلاینت را طرح بزن.

▼ جواب ها

1. (الف) SSG؛ (ب) SSR (سشن/cookies)؛ (ج) SSG + ISR؛ (د) SSG/ISR (عمومی است – داده per-user نیست!).
2. `cache: "no-store"` این fetch را dynamic کرده؛ اگر می خواهی static بماند با تازگی: `next: { revalidate: 60 }`
3. فقط مرحله ای که خروجی مرحله قبلی را می خواهد وابسته است؛ بقیه را با `Promise.all` موازی کن (و وابسته ها را هم می توان با `promise` پاس دادن + `Suspense` استریم کرد).
4. (ا) در Server Component داده روی سرور می ماند و HTML آماده می آید – سریع تر و سئو بهتر؛ (ب) waterfall و `loading-state` دستی حذف می شود + کش/memoization رایگان.
5. سرور: `const { min } = await searchParams` → فیلتر کوئری؛ کلاینت: `router.push(\ ?` input → `(`min=\${v}`) با debounce؛ URL = منبع حقیقت فیلتر.`

→ فصل بعد: Server Actions – نوشتن داده به سبک Next 16.

فصل ۸ – Mutations و Server Actions

منبع رسمی: nextjs.org/docs/app/building-your-application/updating-data – Updating Data 📖

Server Actions and Mutations 🚀 هدف: نوشتن داده بدون API endpoint دستی: تعریف، اتصال به فرم، اعتبارسنجی، اعتبارسنجی مجدد (revalidate) و اتصال به هوک‌های فرم ۱۹ (فصل ۴).

۸.۱ – Server Action: تابع async با "use server"

```
// app/actions.ts
"use server"; // کل فایل = همه توابع سروری

import { revalidatePath } from "next/cache";
import { db } from "@/lib/db";

export async function createPost(formData: FormData) {
  const title = String(formData.get("title") ?? "").trim();
  const body = String(formData.get("body") ?? "").trim();

  if (!title) return { error: "عنوان الزامی است" }; // برگشت به کلاینت

  await db.post.create({ data: { title, body } });

  revalidatePath("/blog"); // کش لیست را باطل کن
  return { ok: true };
}
```

چیزی که اتفاق می‌افتد: Next.js از این تابع یک **RPC endpoint خودکار** می‌سازد – کلاینت فقط **reference** تابع را دارد، بدنه روی سرور اجرا می‌شود (دیتابیس و **secret** امن‌اند).

دو محل تعریف: فایل جدا با **"use server"** بالای فایل، یا **inline** داخل کامپوننت کلاینت (`async function () {` ... `use server";`).

۸.۲ – اتصال به فرم (سه سطح)

سطح ۱ – مستقیم در فرم **Server Component** (ساده‌ترین):

```
export default function NewPostPage() {
  async function addPost(formData: FormData) {
    "use server";
    // ... ذخیره ...
    revalidatePath("/blog");
  }
  return (
    <form action={addPost}>
      <input name="title" required />
      <textarea name="body" />
      <button>ثبت</button>
    </form>
  );
}
```

بدون یک خط progressive enhancement – fetch/state هم دارد (با JS غیرفعال هم کار می‌کند!).

سطح ۲ – با `useActionState`: نتیجه و ارور به کاربر:

```
"use client";
import { useActionState } from "react";
import { createPost } from "@app/actions";

const [state, formAction, isPending] = useActionState(createPost, { error: null });

<form action={formAction}>
  <input name="title" />
  {state.error && <p className="text-red-600">{state.error}</p>}
  <button disabled={isPending}>{isPending ? "... " : "ثبت"}</button>
</form>
```

(امضای اکشن در این حالت: `(prevState, formData)` – پیش‌فرض اول!)

سطح ۳ – بدون فرم: صدا زدن مستقیم:

```
<button onClick={() => deletePost(id)}>حذف</button> // Server Action در onClick ✓
```

(فقط داخل event handler یا transition – نه در حین رندر.)

۸.۳ – اعتبارسنجی مجدد: کش بعد از نوشتن

بعد از هر mutation، داده‌های کش‌شده قدیمی‌اند – باطلشان کن:

```
revalidatePath("/blog"); // کل مسیر
revalidatePath("/blog/[slug]", "page"); // همه صفحات داینامیک
revalidateTag("posts"); // دارند tag "posts" که همه fetch همه
```

ابزار	scope	کی
<code>revalidatePath("/blog")</code>	یک مسیر	تغییر محلی
<code>revalidateTag("posts")</code>	همه‌ی داده‌های برچسب‌خورده	mutation روی یک موجودیت (تمیزترا)
<code>redirect("/blog/42")</code>	ناوبری بعد از اکشن	بعد از ساخت رکورد

با `next: { tags: ["posts"] }` در `fetch` ها (فصل ۷) جفتشان کن – الگوی استاندارد: نوشتن `→ → revalidateTag` UI تازه.

۸.۴ – اعتبارسنجی ورودی: همیشه سرور ★

کلاينت قابل دورزدن است – Server Action خط دفاعی واقعی است:

```
import { z } from "zod";

const PostSchema = z.object({
  title: z.string().min(3).max(120),
  body: z.string().min(1),
});

export async function createPost(prevState: any, formData: FormData) {
  const parsed = PostSchema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) {
    return { error: parsed.error.issues[0].message, values: Object.fromEntries(formData) };
  }
  // ذخیره
}
```

(zod – همان که در دوره Next/وردپرس استفاده کردی؛ در Project واقعی: auth چک + ownership هم همین‌جا.)

۸.۵ – ریزه‌کاری‌های مهم

- **Serializer امن نیست:** Server Action هر ورودی‌ای می‌گیرد – اعتماد نکن؛ `validate + authorization` (مثل `where: { id, userId }` دوره‌های قبل)
- **محدودیت حجم body پیش‌فرض ۱MB** – برای آپلود فایل، `serverActions.bodySizeLimit` یا مسیر Route Handler
- **Sequential:** اکشن‌های یک فرم صف می‌شوند؛ چند اکشن همزمان `→ useTransition` + صدا زدن مستقیم
- **Cookie/redirect در اکشن:** `cookies()` ست می‌شود، `redirect()` وسط اکشن مجاز (خطای `NEXT_REDIRECT` داخلی است، `try/catch` آن را بلع نکن!)

۸.۶ – جانشین‌ها: کی Server Action، کی Route Handler؟

(Route Handler (<code>route.ts</code>	Server Action	
قابل	★ بهترین	فرم‌ها و mutations از UI خود سایت
★ تنها راه	✗	API عمومی (موبایل، webhook، third-party)
★	نه	پاس JSON خام با کنترل هدر

✓ جمع‌بندی فصل

- `"use server"` + تابع `async = endpoint` خودکار امن سمت سرور
- فرم: `useFormStatus/ارور/state` برای `action={serverAction}` → `useActionState` → pending در فرزند
- بعد از نوشتن: `revalidateTag/Path` – وگرنه UI قدیمی می‌ماند
- اعتبارسنجی با `zod` همیشه سرور؛ `auth` هم همین‌جا
- API عمومی = `Route Handler`، نه `Server Action`

📄 تمرین فصل ۸

1. فرم «افزودن محصول» کامل بساز: اکشن سرور + `zod` + ارور + `revalidateTag("products")` + pending – و `fetch` لیست با همان `tag`.
2. چرا `redirect("/blog")` داخل `try { ... } catch { }` اکشن، ارور عجیب می‌دهد؟
3. سناریوی حذف با تأیید: دکمه‌ی حذف در لیست (Server Component) – `Server Action` را چطور صدا می‌زنی و چرا `confirm` را چطور؟
4. مشتری موبایل می‌خواهد همان «ساخت پست» را از اپ خودش بزند – چه تغییر معماری؟

▼ جواب‌ها

2. `redirect` با پرتاب یک خطای خاص `NEXT_REDIRECT` کار می‌کند – `catch` عمومی آن را می‌بلعد. علاج: `redirect` را خارج از `try` بگذار یا در `catch` نوع خطا را `re-throw` کن.
3. حذف = `mutation`: یک کامپوننت کلایینی کوچک `DeleteButton` با `confirm()` + `useTransition` مرورگر، و در `startTransition` صدا زدن `deletePost(id)` (اکشن سروری import شده) + `revalidatePath`.
4. یک `app/api/posts/route.ts` (Route Handler) برای اپ موبایل – همان اعتبارسنجی `zod` را در یک ماژول مشترک بین `Action` و `Handler` نگه دار (DRY).

فصل ۹ – Streaming و مدیریت ارور

منبع رسمی: nextjs.org/docs/app/building-your-application/routing – Loading UI, Error Handling

هدف: صفحه را تکه‌تکه و تدریجی بفرست (Streaming) و برای هر لایه، مرز خطای خودش را بساز –

.loading.tsx ، <Suspense> ، error.tsx ، not-found.tsx

۹.۱ – Streaming: پوسته اول، بقیه بعداً

مشکل: صفحه با دو کوئری – یکی ۵۰ms و یکی ۲ ثانیه. بدون streaming کاربر ۲ ثانیه سفید می‌بیند. جواب: HTML پوسته را فوری بفرست، بخش کند را وقتی حاضر شد تزریق کن:

```
// app/dashboard/page.tsx
import { Suspense } from "react";

export default function DashboardPage() {
  return (
    <div>
      <h1>داشبورد</h1>
      <section>
        <Stats />
      </section>
      <Suspense fallback=<RevenueSkeleton />>
        <Revenue />
      </Suspense>
      <Suspense fallback=<ChartSkeleton />>
        <SlowChart />
      </Suspense>
    </div>
  );
}

async function Revenue() {
  const data = await getRevenue(); // کند - اما بقیه صفحه قفل نمی‌شود
  return <p>{data.total.toLocaleString("fa-IR")}</p>;
}
```

هر Suspense مستقل استریم می‌شود – فالبک هر بخش جدا جایگزین می‌شود. داده کند را پایین ببر و استریم کن؛ کل صفحه را hostage نکن.

۹.۲ – loading.tsx: Suspense خودکار برای کل مسیر

```
// app/dashboard/loading.tsx
export default function Loading() {
  return <DashboardSkeleton />;
}
```

معادل پیچیدن کل `page.tsx` آن مسیر در `Suspense` – در نوبری‌ها فوراً نمایش داده می‌شود. (همچنین `layout.tsx` را ثابت نگه می‌دارد – `nav` گیر نمی‌کند.) می‌توانی در هر عمق بگذاری تا فقط زیرشاخه `skeleton` بگیرد.

💡 `loading.tsx` و `Suspense` دو ابزار یک هدف‌اند: فایل مخصوص = پوشش کل مسیر؛ `Suspense` = دانه‌ریز به دلخواه تو.

۹.۳ – `error.tsx`: مرز خطا

ارور در رندر (کوئری شکست‌خورده و...) → نزدیک‌ترین `error.tsx` آن بخش را نشان می‌دهد و بقیه صفحه زنده می‌ماند:

```
// app/dashboard/error.tsx
"use client"; // اجباری - کامپونت کلاینت ⚠️

export default function DashboardError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void; // تلاش دوباره - رندر همان بخش
}) {
  return (
    <div className="p-6 border border-red-300 rounded-xl bg-red-50">
      <h2>مشکلی پیش آمد</h2>
      <button onClick={reset}>تلاش دوباره</button>
    </div>
  );
}
```

نکته‌ها:

- `error.tsx` **کلاینت** است؛ برای لاگ سمت سرور از `global-error.tsx` یا `onError` استفاده کن – و پیام خام ارور را به کاربر نشان نده (digest برای پشتیبانی)
- `error.tsx` ارورهای **event handler** ها و **Server Action** ها را نمی‌گیرد – آن‌ها را خودت با `try/catch` مدیریت کن
- `global-error.tsx` (در ریشه، کنار layout) = آخرین مرز – باید خودش `<html>` و `<body>` داشته باشد
- `unhandledRejection` در Next 15 + خودش به مرزها می‌رسد (unstable_rethrow برای catch های عمومی اکشن‌ها)

۹.۴ – `notFound(): 404`: کنترل‌شده

```
// در هر Server Component:
import { notFound } from "next/navigation";

const post = await getPost(slug);
if (!post) notFound(); // → نزدیک‌ترین not-found.tsx + status 404

// app/blog/[slug]/not-found.tsx - پیام اختصاصی همان بخش
// app/not-found.tsx - 404 سراسری
```

تفاوت با `throw new Error: notFound` یک «حالت مورد انتظار» است (رکورد نبود) — 404 تمیز می‌دهد، نه صفحه ارور.

۹.۵ – الگوی صفحه production (جمع همه)

```
export default async function ProductPage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;

  const product = await getProduct(id);          // ضروری - بالا
  if (!product) notFound();

  return (
    <main>
      <ProductHeader product={product} />        ← فوری
      <Suspense fallback=<ReviewsSkeleton />>    ← استریم + ارور محلی
        <Reviews id={id} />
      </Suspense>
    </main>
  );
}
```

// error.tsx: ارور فقط بخش را می‌گیرد

✓ جمع‌بندی فصل

- استریم = پوسته فوری + تزریق تدریجی؛ `<Suspense>` دانه‌ریز، `loading.tsx` مسیری
- بخش کند را پایین درخت ببر، کل صفحه را گروگان نگیر
- `error.tsx` = مرز ارور (کلاينت!) + `global-error:reset` آخرین مرز
- ارورهای `event/action` را خودت `catch` کن
- `notFound() = 404` مورد انتظار با پیام اختصاصی

📝 تمرین فصل ۹

1. صفحه با ۴ بخش دارد (هدر، آمار سریع، لیست کند، نمودار بسیار کند) — طرح استریمش را بنویس و بگو کدام‌ها داخل `Suspense` اند و چرا.
2. چرا `error.tsx` باید `"use client"` باشد؟ (راهنمایی: ارور کی و کجا مدیریت می‌شود؟)
3. تفاوت رفتار: `throw` در `Server Component` هنگام رندر، vs پرتاب در `Server Action` هنگام `submit`؟
4. `not-found.tsx` کجاها فعال می‌شود؟ (سه مورد)
5. (کد) یک `loading.tsx` اسکلتی با `CSS` انیمیشن `pulse` برای کارت‌های فیل ۳-ستونه بنویس.

2. چون error.tsx در مرورگر هم اجرا می‌شود (نمایش پیام/دکمه reset در حین ناوبری کلاینتی) – رفتار تعاملی نیاز به کامپوننت کلاینت دارد؛ ارور از سرور serialize شده به آن می‌رسد.
3. رندر → نزدیک‌ترین error.tsx همان بخش؛ Action → به catch کلاینتی صدا زننده (useActionState state.error) می‌رسد، مرز error.tsx آن را نمی‌گیرد.
4. (۱) فراخوانی notFound() در همان بخش؛ (۲) مسیری که match نشد؛ (۳) URL دستکاری‌شده خارج از پارامترهای مجاز – همه به نزدیک‌ترین not-found.tsx می‌روند.
- 5.

```
export default function Loading() {
  return (
    <div className="grid grid-cols-3 gap-4 animate-pulse">
      {Array.from({ length: 6 }).map((_, i) => (
        <div key={i} className="h-40 bg-gray-200 rounded-xl" />
      ))}
    </div>
  );
}
```

فصل ۱۰ – بهینه‌سازی: Image، Font، Metadata

منبع رسمی: nextjs.org/docs/app/building-your-application/optimizing – Optimizing بخش‌ها  هدف: سه بهینه‌ساز داخلی که امتیاز Lighthouse و Core Web Vitals تو را می‌سازند: next/image، next/font و Metadata API.

۱۰.۱ – next/image: تصویر هوشمند

```
import Image from "next/image";
import hero from "@public/hero.jpg"; // import = اندازه خودکار!

// Static import (پیشنهادی - سایز خودش می‌داند):
<Image src={hero} alt="صفحه اصلی" priority placeholder="blur" />

// Remote:
<Image src="https://cdn.site.com/pic.jpg"
  alt="محصول" width={800} height={450}
  sizes="(max-width: 768px) 100vw, 50vw" />
```

چه چیزهایی مجانی می‌گیری:

قابلیت	اثر
بهینه‌سازی/فرمت مدرن (WebP/AVIF) + resize on demand	حجم چند برابر کمتر
Lazy load پیش‌فرض	صفحه اول سبک
جلوگیری از layout shift (نیاز به width/height یا fill)	CLS صفر
priority	Preload برای LCP (تصویر اول صفحه!)

پرچم‌های مهم:

- **priority** – روی تصویر بالای فولد (hero) بزن؛ باقی lazy می‌مانند
 - **sizes** – به مرورگر بگو تصویر چه عرضی دارد تا مناسب‌ترین نسخه را بگیرد؛ بدون آن، تصویر بزرگ‌تر از نیاز دانلود می‌شود
 - **fill** + پدر با **position:relative** – برای تصاویر full-width با object-fit
 - **placeholder="blur"** – فقط با static import (blurDataURL خودکار)
- عکس‌های دامنه خارجی → **remotePatterns** در next.config (فصل ۱۲ وردپرس دیدی!).

اصل طلایی: تصویر اول (LCP) = `sizes` + `priority` درست؛ بقیه = `lazy` پیش فرض.

۱۰.۲ — next/font: فونت بدون CLS و بدون درخواست اضافه

```
// app/layout.tsx
import { Vazirmatn } from "next/font/google";

const vazir = Vazirmatn({
  subsets: ["arabic"],
  variable: "--font-vazir",
  display: "swap",
});

export default function RootLayout({ children }) {
  return (
    <html lang="fa" dir="rtl" className={vazir.variable}>
      <body>{children}</body>
    </html>
  );
}
```

```
/* tailwind/css */
font-family: var(--font-vazir);
```

مزایا نسبت به `<link>` گوگل فونت:

- فونت در **build** دانلود و **self-host** می‌شود — بدون درخواست به گوگل در `runtime` (حریم + سرعت + بدون FOUT کلاسیک)
- `display: swap` — متن فوری با فونت جایگزین، بعد تعویض
- `variable: --font` — مصرف تمیز در Tailwind/CSS
- فایل‌های لوکال هم: `next/font/local`

```
// static:
export const metadata: Metadata = {
  title: "فروشگاه دیجی",
  description: "بازار gadgets بهترین",
  openGraph: { images: ["/og.png"] },
};

// dynamic - جزئیات در صفحه:
export async function generateMetadata({ params }): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPost(slug);
  return {
    title: post.title,
    description: post.excerpt,
    alternates: { canonical: `/blog/${slug}` },
    openGraph: {
      title: post.title,
      images: post.featuredImage?.sourceUrl ? [post.featuredImage.sourceUrl] : [],
    },
  };
}
```

- metadata ارزیابی می‌شود و در `<head>` تزریق می‌شود – با سرور هماهنگ
- قالب‌بندی با `title.template` در layout والد: "دیجی | %s" – فرزند فقط title بدهد
- فایل‌های قراردادی: `app/icon.png` (فاویکون)، `app/opengraph-image.tsx` (OG تصویر تولیدی با کدا)، `sitemap.ts`، `robots.ts`

```
// app/sitemap.ts - سایت‌مپ داینامیک از دیتابیس:
export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const posts = await getPosts();
  return [
    { url: "https://site.com", changeFrequency: "daily", priority: 1 },
    ...posts.map((p) => ({ url: `https://site.com/blog/${p.slug}`, lastModified: p.date })),
  ];
}
```

(این دقیقاً همان چیزی است که در دوره وردپرس Headless برای سئو استفاده کردی – حالا رسمی‌اش را می‌دانی.)

۱۰.۴ – بهینه‌سازی‌های دیگر (یک نگاه)

ابزار	کار
next/script	بارگذاری اسکریپت third-party با استراتژی (<code>afterInteractive</code>)
next/dynamic	import lazy کامپوننت (کد سنگین مثل ادیتور) – <code>ssr: false</code> برای browser-only
Bundle Analyzer	@next/bundle-analyzer – کی باندل سنگین شد؟
experimental.optimizePackageImports	tree-shake بهتر آیکون لایبرری‌ها

✓ جمع‌بندی فصل

- Image: بهینه‌سازی + CLS + lazy صفر؛ hero → priority/sizes؛ خارجی → remotePatterns
- Font: self-host در build با next/font – بدون درخواست خارجی، با swap
- Metadata: static/dynamic + template + فایل‌های قراردادی (icon/og/sitemap/robots)
- dynamic import برای کد سنگین؛ اسکریپت‌ها با next/script

📝 تمرین فصل ۱۰

1. صفحه‌ای با یک hero بزرگ بالای فولد و ۱۲ کارت عکس‌دار – هر تصویر چه تنظیمی می‌خواهد؟ (priority/sizes/fill?)
2. چرا `<Image src="https://cdn.x.com/a.jpg" />` بدون width/height یا fill ارور می‌دهد؟ این اجبار چه چیزی را تضمین می‌کند؟
3. تفاوت `display: swap` و فونت بدون swap از نظر تجربه کاربر و CLS؟
4. `generateMetadata` بنویس که title را از قالب والد بگیرد (شاپ | %s) و og:image از تصویر شاخص پست.
5. (تحقیق) `opengraph-image.tsx` را امتحان کن – تصویر اشتراک‌گذاری توئیتر را با کد تولید کن (ImageResponse).

1. priority (lazy) + بدون: کارت‌ها: (اگر static import) hero: `priority + sizes="100vw"` + placeholder
`relative` یا `fill` با پدر `sizes="(max-width: 768px) 100vw, 33vw"` + width/height
2. اجبار به ابعاد = جلوگیری از layout shift (CLS) – مرورگر از قبل فضا رزرو می‌کند؛ با `fill` هم پدر باید ابعاد داشته باشد.
3. swap: متن فوری با فونت سیستمی سپس تعویض (FOUT – متن همیشه دیده می‌شود)؛ بدون swap: صفحه تا لود فونت متن را مخفی می‌کند (FOIT) – CLS و تأخیر ادراکی بدتر.
- 4.

```
export async function generateMetadata({ params }): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPost(slug);
  return {
    title: post.title, // والد template با
    openGraph: { images: post.image ? [post.image] : [] },
  };
}
```

فصل ۱۱ – Deployment و Route Handler، Middleware

منبع رسمی: nextjs.org/docs/app/building-your-application/routing/route-handlers + 

Middleware + Deploying  هدف: لایه API خود (Next.js (Route Handler)، لایه بین‌راهی (Middleware)، متغیرهای محیطی با دو دنیای سرور/کلاينت، و نکات دپلوی Vercel/self-host.

۱۱.۱ – API Route Handler داخل خود Next

```
// app/api/products/route.ts
import { NextRequest, NextResponse } from "next/server";

export async function GET(req: NextRequest) {
  const q = req.nextUrl.searchParams.get("q") ?? "";
  const products = await db.product.findMany({ where: { title: { contains: q } } });
  return NextResponse.json(products);
}

export async function POST(req: NextRequest) {
  const body = await req.json();
  const product = await db.product.create({ data: body });
  return NextResponse.json(product, { status: 201 });
}

// app/api/products/[id]/route.ts: و دینامیک
export async function DELETE(req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  const { id } = await params; // ⚠️ باز هم Promise!
  await db.product.delete({ where: { id: Number(id) } });
  return new NextResponse(null, { status: 204 });
}
```

- همان الگوی Express-ish ولی فایل‌محور؛ متدهای HTTP های هم‌نام
- وابستگی‌های `req.nextUrl`، `request`، `cookies`، `headers`،
- Caching:** GET به‌صورت پیش‌فرض در build کش می‌شود اگر static باشد – برای dynamic شدن: از `req` استفاده کن یا `export const dynamic = "force-dynamic"` (یا در fetch های داخلی (no-store)
- Stream و `await req.text()` Webhook: برای امضای webhook ها (مثل درگاه پرداخت)

کی Route Handler و کی Server Action؟ (تکرار از فصل ۸ – یک‌خطی)

مصرف‌کننده خارج از مرورگر خودت (موبایل/third-party/webhook) → Handler؛ فرم و UI خود سایت → Server Action.

۱۱.۲ – Middleware: نگهبان بین‌راه

در ریشه — قبل از هر request (و completion بعد از آن) اجرا می‌شود؛ روی edge سبک است. ⚠️
به‌روزرسانی **Next.js 16**: نام این فایل به `proxy.ts` تغییر کرده و به‌طور پیش‌فرض روی **Node.js runtime** اجرا می‌شود؛
فایل قدیمی `middleware.ts` هنوز پشتیبانی و مثل قبل روی edge اجرا می‌شود — مفهوم «edge» را در **فصل ۱۳** عمیق
بخوان:

```
import { NextRequest, NextResponse } from "next/server";

export function middleware(req: NextRequest) {
  const session = req.cookies.get("session");

  // محافظت از مسیرها:
  if (!session && req.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", req.url));
  }

  // افزودن هدر:
  const res = NextResponse.next();
  res.headers.set("x-frame-options", "DENY");
  return res;
}

export const config = {
  matcher: ["/dashboard/:path*", "/admin/:path*"], // فقط این مسیرها
};
```

کاربردهای متعارف: A/B، auth-redirect، rewrite، افزودن هدرهای امنیتی، i18n (تشخیص زبان). ⚠️ سنگین اجرا نکن
— روی هر request می‌آید؛ logic اتصال دیتابیس اینجا ممنوع.

۱۱.۳ – Environment Variables: دو دنیا

```
# .env.local
DATABASE_URL="postgresql://..." # فقط سرور ✓
STRIPE_SECRET="sk_..." # فقط سرور ✓
NEXT_PUBLIC_API_URL="https://api.site.com" # به کلاینت هم می‌رود ⚠️
```

پیشوند	کجا در دسترس	خطر
بدون پیشوند	فقط Server Component / Action / Route Handler	امن برای رازها
NEXT_PUBLIC_	+ باندل کلاینت	راز هرگز! چون داخل JS ساخته‌شده embed می‌شود

قواعد: `.env*` در `gitignore`؛ روی Vercel/سرور از پنل Environment Variables؛ `process.env` فقط سمت سرور در
زمان اجرا خوانده می‌شود (NEXT_PUBLIC ها در build اینلاین می‌شوند).

Vercel (ساده‌ترین و کامل‌ترین برای App Router)

`npx vercel` در پنل *GitHub* از *import*: یا #

- هر push به production deploy → main؛ هر PR → preview URL
- ISR/Caching/Image Optimization همه managed
- Env vars را در پنل ست کن (!Local ≠ production)
- Server Actions/SSR نیازمند serverless functions – خودکار

Self-host (سرور خودت – دوره Docker!)

```
# سرور:
docker run -d -p 3000:3000 --restart unless-stopped \
  -e DATABASE_URL="..." myapp:1.0
# و دامنه؛ با استانداردهای فصل ۱۲ دوره داکر SSL برای nginx/caddy پشت:
# healthcheck، بکاپ دیتابیس، prune، log ها
```

نکته self-host: ISR/Cache روی دیسک/Redis؛ Image Optimization تنظیم می‌خواهد (`images.unoptimized`) یا سرویس جدا) – Vercel این‌ها را رایگان حل کرده؛ انتخاب architecture.

۱۱.۵ – چک‌لیست Production

- [] env های production در پنل پلتفرم؛ هیچ رازی با `_NEXT_PUBLIC`
- [] `metadataBase` ست شده (برای URL های OG)
- [] ارورها مانیتور می‌شوند (Sentry و...) – digest ها بی‌معنا بدون لاگ
- [] `middleware` matcher دقیق – نه catch-all سنگین
- [] API ها: rate limit + auth (فصل ۸)
- [] تست: build لوکال `next build` بدون ارور ESLint/type قبل از push

جمع‌بندی فصل ✓

- API = Route Handler فایل‌محور؛ Server Action برای UI خودت، Handler برای دنیای بیرون
- Middleware = نگهبان سبک edge با matcher دقیق (auth/هدر)
- env: بدون پیشوند = سرور؛ `_NEXT_PUBLIC` = در باندل کلاینت embed – رز هرگز

- Vercel = صفر پیکربندی؛ self-host = داکر + nginx + مسئولیت ISR/cache

تمرین فصل ۱۱

1. GET /api/orders?status=paid با فیلتر و pagination (هدر X-Total-Count) بنویس – و بگو چرا اینجا Server Action جوابگو نبود.
2. Middleware بنویس: کاربر بدون کوکی session از /admin/* به <مسیر فعلی>/login?next= برود و بعد از لاگین برگردد.
3. چرا NEXT_PUBLIC_DB_PASSWORD یک فاجعه است حتی اگر «فقط برای تست» باشد؟
4. مقایسه دپلوی: پروژه‌ات را به Vercel ببر (env ها را یادت نرود) – یا اگر VPS داری، با دانش داکترت طرح self-host بنویس (نه کد – معماری).

▼ جواب‌ها

1. Server Action برای مصرف‌کننده‌های خارج از اپ (موبایل/webhook) قابل فراخوانی مستقیم نیست و کنترل هدر/status کاستوم ندارد – Handler استاندارد API عمومی است.
- 2.

```
export function middleware(req: NextRequest) {
  if (!req.cookies.get("session")) {
    const url = new URL("/login", req.url);
    url.searchParams.set("next", req.nextUrl.pathname);
    return NextResponse.redirect(url);
  }
}
export const config = { matcher: ["/admin/:path*"] };
```

3. چون همه چیز NEXT_PUBLIC_ در فایل‌های JS ساخته شده embed می‌شود – هرکسی View Source بزند رمز دیتابیس را می‌بیند؛ «تست» هم یک روز production می‌شود.
4. Vercel: import ریپو + env ها؛ image Self-host: اپ + network داکر به دیتابیس + caddy برای SSL + backup revalidate + cron روی دیسک – مسئولیت‌های Vercel را خودت می‌پذیری.

→ فصل آخر: چیت‌شیت دوتایی + گلاساری + ۲۰ سوال مصاحبه.

فصل ۱۲ – چیت‌شیت دوتایی، گلاساری و ۲۰ سوال مصاحبه

🎯 هدف: فشرده‌ترین مرجع ممکن – React در یک صفحه، Next.js در یک صفحه، و سوالاتی که در مصاحبه‌های Next.js واقعاً پرسیده می‌شوند.

۱۲.۱ – چیت‌شیت React 19

```
// == State ==
const [count, setCount] = useState(0);
setCount((c) => c + 1); // آپدیت وابسته → فرم تابعی
setUser({ ...user, name: "Ali" }); // immutable همیشه
const [state, dispatch] = useReducer(reducer, init);

// == Refs ==
const ref = useRef<HTMLInputElement>(null);
ref.current?.focus(); // تغییر = بدون رندر

// == Effects (فقط همگام‌سازی بیرون) ==
useEffect(() => {
  const conn = connect(id);
  return () => conn.disconnect(); // cleanup همیشه
}, [id]); // وابستگی‌ها صادقانه

// == Memo (اندازه‌گیری‌شده) ==
const v = useMemo(() => heavy(a), [a]);
const fn = useCallback(() => {...}, []);

// == Concurrent ==
const [isPending, start] = useTransition();
start(() => setFilter(q)); // کم‌اولویت
const deferred = useDeferredValue(value);

// == فرم‌ها (React 19) ==
const [state, action, pending] = useActionState(fn, init);
const { pending } = useFormStatus(); // <form> داخل
const [opt, addOpt] = useOptimistic(list, (c, x) => [...c, x]);
const data = use(promise); // حتی در شرط

// == قوانین طلایی ==
// state محاسبه‌شدنی = محاسبه نه state
// شناسه پایدار = key نشود؛ mutate هرگز
// {0 && <X/>} تله دارد → {n > 0 && ...}
```

```
// == فایل‌های قراردادی ==
page.tsx / layout.tsx / template.tsx
loading.tsx (Suspense خودکار) / error.tsx ("use client")
not-found.tsx / route.ts (API بدون page)
global-error.tsx (با <html>)
```

```
// == پارامترها (Next 15+: Promise!) ==
const { slug } = await params; // [slug]
const { q } = await searchParams; // ?q= (dynamic می‌کند)
```

```
// == استراتژی رندر ==
export const revalidate = 60; // ISR
export async function generateStaticParams() {...} // SSG دینامیک
await cookies(); await headers(); // → SSR dynamic
fetch(url, { next: { revalidate: 60, tags: ["posts"] } });
fetch(url, { cache: "no-store" });
```

```
// == Mutations ==
"use server";
export async function createPost(prev, formData) {
  const parsed = Schema.safeParse(...);
  if (!parsed.success) return { error: "..." };
  await db.create(...);
  revalidateTag("posts"); // یا revalidatePath("/blog")
  redirect("/blog");
}
// فرم: <form action={formAction}> + useActionState + useFormStatus
```

```
// == Streaming و ارور ==
<Suspense fallback={<Skeleton />}><Slow /></Suspense>
notFound(); // → not-found.tsx
// error.tsx: {error, reset} - reset = تلاش دوباره
```

```
// == بهینه‌سازی ==
<Image src={img} alt="" priority sizes="..." /> // LCP
const f = Vazirmatn({ subsets: ["arabic"], variable: "--f" });
export const metadata / generateMetadata
```

```
// == API و میان‌افزار ==
export async function GET(req: NextRequest) {...} // route.ts
middleware.ts + config.matcher // auth/هدر
// env: بدون پیشنهاد=سرور؛ NEXT_PUBLIC_ = (! بدون راز) کلاینت
```

```
// == Server vs Client ==
// مرز یک‌طرفه به پایین "use client" = پیش‌فرض سرور؛
// تابع از مرز رد نمی‌شود → Server Action
// سروری children کلاینی با Provider
```

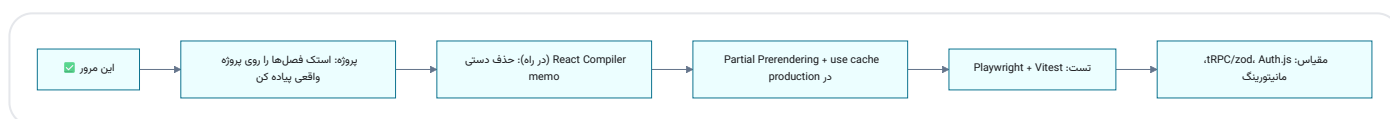
اصطلاح	یعنی
RSC (Server Component)	کامپوننت فقط سروری – خروجی payload به کلاینت
Client Component	SSR اولیه + hydration + تعامل
Hydration	زنده شدن HTML سرور با JS کلاینت
RSC Payload	فرمت داده‌ای خروجی RSC برای ناوبری/استریم
SSG / ISR / SSR / CSR	استاتیک بیلدی / استاتیک با بازتولید / هر درخواست / مرورگر
Server Action	تابع async سروری قابل صدا از کلاینت (RPC)
Revalidation	ابطال کش – با زمان (ISR)، مسیر یا تگ
Request Memoization	یک بار fetch برای URL یکسان در یک رندر
Streaming	ارسال تدریجی HTML با Suspense
Route Handler	endpoint در route.ts
Middleware	کد بین‌راهی edge قبل از مسیر
Prefetch	پیش‌بارگذاری مسیر بعدی با Link
Waterfall	زنجیره await های پشت‌سرهم – دشمن سرعت
Prop Drilling	پاس‌دادن props از لایه‌های واسط → Context/Composition

۱۲.۴ – ۲۰ سوال مصاحبه (با پاسخ کوتاه)

1. **React: چرا key؟** تطبیق آیتم‌ها بین رندها برای حفظ state/DOM؛ index فقط برای لیست‌های کاملاً ثابت. 2. **state** چرا **snapshot** است؟ هر رندر نسخه ثابت متغیرها را می‌بیند؛ آپدیت وابسته → فرم تابعی. 3. **چرا immutable؟** React تغییر را با مرجع تشخیص می‌دهد؛ mutation = rerender نمی‌شود. 4. **useEffect دقیقاً برای چیست؟** همگام‌سازی با سیستم بیرونی – نه lifecycle، نه محاسبه، نه **cleanup**. 5. **event کی اجرا می‌شود؟** قبل از هر اجرای دوباره effect و قبل از unmount – قطع همگام‌سازی قبلی. 6. **useMemo کی؟** محاسبه سنگین اندازه‌گیری شده یا مرجع پایدار برای وابستگی/مصرف memo شده. 7. **useTransition چه مشکلی را حل می‌کند؟** آپدیت کند UI فوری را قفل نمی‌کند (قابل قطع است). 8. **use vs useContext؟** use در شرط/حلقه مجاز است و promise هم می‌خواند. 9. **Server Component Next.js: چیست؟** کد فقط سروری؛ JS به کلاینت نمی‌رود؛ db/secret دارد؛ hook ندارد. 10. **"use client" دقیقاً چه می‌کند؟** از آن نقطه به پایین درخت را مرز کلاینت می‌کند (import ها هم). 11. **چرا Client**

Component هم SSR می‌شود؟ HTML اولیه از سرور می‌آید، بعد interactivity – hydrate بعدی کلاینتی است. 12. چه چیزی از مرز سرور → کلاینت رد نمی‌شود؟ توابع (به جز Server Actions) و آبجکت‌های با 13. behavior. فرق **SSG/ISR/SSR**؟ build یک بار / build + بازتولید دوره‌ای / هر درخواست. 14. چطور صفحه dynamic می‌شود؟ **Route Handler** یا cookies/headers/searchParams با **Server Action**. 15. no-store چیست و چه فرقی با **Route Handler**؟ RPC برای UI خودت؛ API = Handler عمومی با کنترل هدر/status. 16. بعد از mutation داده چطور تازه می‌شود؟ revalidateTag/Path – کش مسیر/تگ باطل می‌شود. 17. **Streaming** چه مشکلی را حل می‌کند؟ بخش کند صفحه بقیه را گروگان نمی‌گیرد – پوسته فوری + تزریق. 18. **params** چرا **Promise** است؟ مدل 15 Next + برای سازگاری استریم/async – با **env**. 19. **await** چطور امن می‌ماند؟ بدون پیشوند فقط سرور؛ **NEXT_PUBLIC** در باندل embed می‌شود – راز هرگز. 20. **template** و **layout**؟ **layout** بین ناوبری‌ها حفظ می‌شود؛ **template** هر بار remount (برای انیمیشن ورود).

۱۲.۵ – نقشه راه پس از این مرور



جمع‌بندی نهایی: این مرور را هر ۲-۳ ماه یک بار برو – داکيومنت Next.js سریع تغییر می‌کند (مثل سیستم کش ۱۵→۱۶)، ولی مدل‌های ذهنی این دوره (مرز سرور/کلاینت، استراتژی رندر، قوانین state) پایدارند. ترکیبش با ۵ دوره قبلیت (دیتابیس، گیت، CLI، داکر، وردپرس) = پروفایل کامل فول‌استک. موفق باشی! 🧠 ▲

فصل ۱۳ – عمق‌سنجی: ۹ سوال کلیدی

📖 منابع رسمی: [react.dev/reference/react](https://react.dev/reference/react-use) – use, useActionState, useFormStatus, useOptimistic و nextjs.org/docs – RSC, Middleware 🍯 هدف: فصل‌های قبل هر کدام یک قاب کوتاه از این موضوعات را داشتند؛ اینجا هر ۹ سوال پرتکرار را عمیق، چندسناریویی و با تله‌ها باز می‌کنیم. مکمل فصل‌های ۴، ۶، ۸، ۹ و ۱۱ است.

۱۳.۱ – هوک use با مثال‌های واقعی

use دقیقاً چیست؟

use یک API برای «خواندن مقدار یک Resource» است. فقط دو نوع Resource می‌فهمد:

1. **Promise** → تا promise حل شود، کامپوننت را **suspend** می‌کند (نزدیک‌ترین `<Suspense>` فعال می‌شود)

2. **Context** → مثل `useContext` مقدار کانتکست را می‌دهد

دو تفاوت بنیادی با بقیه هوک‌ها:

use	هوک‌های معمولی (useState و ...)
داخل شرط، حلقه و return های زود هنگام هم مجاز	فقط در بالاترین سطح کامپوننت
مثل یک تابع معمولی است؛ فقط باید داخل کامپوننت/هوک سفارشی صدا زده شود	قانون order-based دارد

سناریو ۱ – Promise از Server Component (رایج‌ترین الگو در Next.js)

کلید الگو: در سرور promise را بدون `await` می‌سازی و به کلاینت می‌فرستی. خود promise از مرز سرور → کلاینت قابل انتقال است و کلاینت با `use` منتظرش می‌ماند:

```
// app/dashboard/page.tsx - Server Component
import { Suspense } from "react";
import Profile from "../Profile";
import ActivityFeed from "../ActivityFeed";

export default function Page() {
  const userPromise = getUser(1); // ⚠️ await نکن - فقط promise را پاس بده
  const activityPromise = getRecentActivity(1);

  return (
    <div>
      <Suspense fallback={<ProfileSkeleton />}>
        <Profile userPromise={userPromise} />
      </Suspense>
      <Suspense fallback={<ActivitySkeleton />}>
        <ActivityFeed activityPromise={activityPromise} />
      </Suspense>
    </div>
  );
}
```

```
// Profile.tsx - Client Component
"use client";
import { use } from "react";
import type { User } from "../types";

export default function Profile({ userPromise }: { userPromise: Promise<User> }) {
  const user = use(userPromise); // می‌شود suspend همین کامپوننت، promise تا حل شدن
  return <h1 className="text-2xl font-bold">{user.name}</h1>;
}
```

چرا این الگو عالی است؟ دو درخواست موازی اجرا می‌شوند و هر کدام که زودتر رسید، همان بخش استریم می‌شود (اتصال به فصل ۹). اگر `await` می‌کردی، کل صفحه تا کندترین query صبر می‌کرد.

سناریو ۲ - خواندن شرطی Context (مهم‌ترین برتری `use` بر `useContext`)

```
"use client";

function CouponBanner() {
  const { plan } = use(UserContext); // می‌شود با شرط

  if (plan !== "free") return null; // کاربران پولی اصلاً به کانتکست تخفیف نیاز ندارند
  const coupon = use(CouponContext); // این خط ارور قواعد هوک می‌گرفت با useContext ✓
  return <p>کد تخفیف تو: {coupon.code}</p>;
}
```

سناریو ۳ - ارور هم خودکار به `ErrorBoundary` می‌رود

`use(promise)` اگر `reject` promise شود، ارور را به نزدیک‌ترین `Error Boundary` می‌اندازد - بدون `try/catch`:

```
<ErrorBoundary fallback={<RetryButton />}>
  <Suspense fallback={<Spinner />}>
    <Profile userPromise={userPromise} />
  </Suspense>
</ErrorBoundary>
```

یعنی «بارگذاری» و «شکست» هر دو به صورت declarative مدیریت می‌شوند.

سناریو ۴ – ⚠ تله‌ی بزرگ: promise تازه در هر رندر

`use` خودش promise را `cache` نمی‌کند. اگر در هر رندر promise جدید بسازی، هر بار `suspend` می‌شوی و حلقه‌ی «اسپینر بی‌نهایت» می‌گیری:

```
// ❌ دائمی suspend → جدید promise غلط - هر رندر
function Profile({ id }: { id: number }) {
  const user = use(fetchUser(id));
  return <h1>{user.name}</h1>;
}
```

راه‌های درست، به ترتیب ترجیح:

1. `promise` را از والد/سرور بگیر (سناریو ۱) – بهترین راه

2. در سرور، با `cache()` از react کش کن: `tsx import { cache } from "react"; const getUser = cache((id: number) => db.user.findUnique({ where: { id } }));`

3. در کلاینت، `promise` را در `state` یا `event` نگه دار، نه ساخت مستقیم در رندر؛ برای `fetch` سمت کلاینت به‌هرحال `SWR` یا `TanStack Query` ابزار درست‌اند

4. در مازول (خارج کامپوننت) `promise`/کش بساز – فقط اگر داده `global` است

جدول جمع‌بندی ۱۳.۱

می‌خواهی...	ابزار درست
خواندن داده در Server Component	<code>await</code> معمولی
دادن داده از سرور به کلاینت با <code>Suspense</code>	<code>promise</code> بساز و پاس بده + <code>use</code> در کلاینت
خواندن شرطی کانتکست	<code>use(Context)</code>
<code>fetch</code> سمت کلاینت با کش و <code>refetch</code>	<code>SWR</code> / <code>TanStack Query</code>
خواندن یک‌باره در رندر سرور + اشتراک بین چند کامپوننت	<code>await</code> + <code>cache()</code>

۱۳.۲ – `useActionState`: چرخه کامل فرم + Server Action

امضای هوک

```
const [state, formAction, isPending] = useActionState(action, initialState, permalink?);
```

- **action**: تابعی با امضای متفاوت از action معمولی – آرگومان اول `prevState` است: `(prevState, formData)` `newState` (همگام یا `async`)
- **state**: خروجی آخرین اجرای `action`؛ در اولین رندر `initialState` است
- **formAction**: همان `action` است که مستقیم به `<form action={...}>` می‌دهی
- **isPending**: از شروع `submit` تا پایان اجرای اکشن `true`
- **permalink**: (پیشرفته) مسیر جایگزین برای وقتی JS هنوز لود نشده – به ندرت لازم می‌شود

چرا `prevState` وجود دارد؟

چون `action` ممکن است چند بار اجرا شود (هر `submit`). فرم تو باید بداند «اجرای قبلی چه نتیجه‌ای داد» تا مثلاً ارور قبلی را نگه دارد، پیام‌ها را انباشته کند یا سرشمارش کند:

```
const [state, formAction, isPending] = useActionState(
  async (prevState, formData) => {
    const email = String(formData.get("email") ?? "");

    if (!email.includes("@")) {
      return { ...prevState, error: "ایمیل نامعتبر است", attempts: prevState.attempts + 1 };
    }
    await addToNewsletter(email);
    return { error: null, success: true, attempts: prevState.attempts + 1 };
  },
  { error: null, success: false, attempts: 0 }
);
```

مثال کامل Server Action – Next.js + اعتبارسنجی + نتیجه

```
// app/actions.ts
"use server";

export type SignUpState = { error: string | null; ok: boolean };

export async function signUp(prevState: SignUpState, formData: FormData): Promise<SignUpState> {
  const email = String(formData.get("email") ?? "").trim();

  if (!email.includes("@")) {
    return { error: "ایمیل نامعتبر است", ok: false }; // ← به state می‌رود، نه ارور
  }
  if (await emailExists(email)) {
    return { error: "این ایمیل قبلاً ثبت شده", ok: false };
  }
  await db.user.create({ data: { email } });
  return { error: null, ok: true };
}
```

```
// app/signup-form.tsx
"use client";

import { useActionState } from "react";
import { signUp } from "../actions";

export default function SignupForm() {
  const [state, formAction, isPending] = useActionState(signUp, { error: null, ok: false });

  return (
    <form action={formAction}>
      <input name="email" type="email" placeholder="ایمیل" />
      <button disabled={isPending}>
        {isPending ? "ثبت نام" : "در حال ثبت..."}
      </button>
      {state.error && <p className="text-red-600">{state.error}</p>}
      {state.ok && <p className="text-green-600">ثبت شد! 🎉</p>}
    </form>
  );
}
```

چهار تله مهم

1. **throw** به **state** نمی‌رود — اگر داخل action ارور بندازی، به Error Boundary می‌رود و فرم کاربر با «ارور قرمز فرم» جور در نمی‌آید. برای خطای قابل نمایش به کاربر، **return** کن؛ throw فقط برای باگ‌های واقعی.
2. **redirect()** را **catch** نکن — **redirect** با انداختن یک ارور خاص کار می‌کند؛ اگر دورش try/catch بگذاری، ناپرویی خراب می‌شود.
3. اسم قدیمی: در 14 Next.js و 18 React این هوک **useFormState** بود؛ در 19 React به **useActionState** تغییر نام داد (و از **react-dom** به **react** منتقل شد).
4. آرگومان اضافه لازم داری؟ با `bind : tsx const updateWithId = updateProfile.bind(null, userId); //`
امضای اکشن `const [state, formAction] = useActionState(updateWithId, null); //` آرگومان اول `userId =`
`(userId, prevState, formData) ≥ ...`

💡 چون action یک Server Action است، حتی قبل از لود شدن JS هم فرم کار می‌کند (progressive enhancement) — Browser بدون JS فرم را به صورت POST معمولی می‌فرستد و سرور نتیجه را رندر می‌کند.

۱۳.۳ — useFormStatus : وضعیت فرم پدر، از داخل فرزند

مسئله‌ای که حل می‌کند

دکمه‌ی submit معمولاً یک کامپوننت مشترک و جداست و به state فرم دسترسی ندارد. راه قدیمی = prop drilling یا Context دستی. راه جدید: فرزند، خودش وضعیت نزدیک‌ترین `<form>` والد را می‌خواند.

```

"use client";

import { useFormStatus } from "react-dom"; // ⚠️ از react-dom نه react است
export function SubmitButton({ children }: { children: React.ReactNode }) {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending}
      className={pending ? "opacity-50 cursor-not-allowed" : ""}>
      {pending ? "⌚ در حال ارسال" : children}
    </button>
  );
}

```

```

// استفاده - یک دکمه، در هر تعداد فرم:
<form action={signIn}>
  <input name="email" />
  <SubmitButton>ثبت نام</SubmitButton> { /* باید داخل <form> باشد */ }
</form>

<form action={deleteAccount}>
  <SubmitButton>حذف حساب</SubmitButton>
</form>

```

خروجی کامل هوک

```
const { pending, data, method, action } = useFormStatus();
```

فیلد	معنا
pending	اکشن فرم در حال اجراست؟
data	همان FormData در حال submit (مثلا برای نمایش فیلدها هنگام ارسال)
method	"post" یا "get"
action	تابع action که به فرم داده شده

تله‌ها

- باید در درختِ رندر، داخل `<form>` باشد. اگر هوک را در کامپوننتی صدا بزنی که خودش `<form>` را رندر می‌کند (نه فرزندش)، همیشه `pending: false` می‌بینی — چون «والد» یعنی پدر/اجداد در درخت رندر، نه کامپوننت بیرونی.
- فقط حین submit معنا دارد؛ برای نتیجه/ارور (state) وسیله نیست — آن کار `useActionState` است.

نیاز	ابزار
دکمه/اسپینر داخل فرم بخواهد pending بداند	<code>useFormStatus</code>
فرم بخواهد نتیجه/ارور اکشن را نمایش دهد	<code>useActionState</code>
هر دو	با هم ترکیب می‌شوند بدون هیچ prop ای

۱۳.۴ – `useOptimistic` : به‌روزرسانی خوش‌بینانه

مدل ذهنی

«اول UI را نشان بده که امیدواریم بعد از موفقیت این‌طوری شود؛ درخواست واقعی را در پس‌زمینه بفرست؛ وقتی جواب واقعی آمد، React خودش حالت خوش‌بینانه را دور می‌ریزد و با state واقعی جایگزین می‌کند.»

```
const [optimisticState, addOptimistic] = useOptimistic(
  state, // state واقعی (از props یا useState)
  (current, input) => { // reducer - باید pure خوش‌بینانه باشد
    nextState(current, input)
  }
);
```

- `optimisticState` : نسخه‌ی موقت برای نمایش
- `addOptimistic(value)` : یک «به‌روزرسانی خوش‌بینانه» ثبت می‌کند؛ باید داخل یک **action** یا **transition** صدا زده شود

مثال کامل – چت با Server Action در Next.js

```
// app/actions.ts
"use server";

export async function sendMessage(formData: FormData) {
  const text = String(formData.get("text") ?? "");
  await db.message.create({ data: { text } });
  revalidatePath("/chat"); // ← state واقعی بعد از این به کلاینت می‌رسد
}
```

```
// app/chat/messages.tsx
"use client";

import { useOptimistic, useRef } from "react";
import { sendMessage } from "../actions";

type Msg = { id: string; text: string; sending?: boolean };

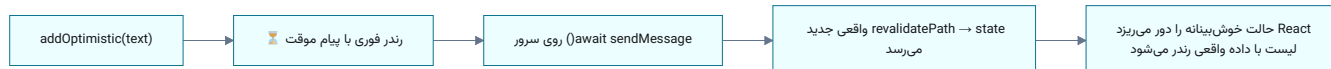
export default function Messages({ messages }: { messages: Msg[] }) {
  const [optimisticMsgs, addOptimistic] = useOptimistic(
    messages,
    (current, text: string) => [
      ...current,
      { id: `optimistic-${Date.now()}`, text, sending: true }, // پیام موقت
    ]
  );

  const formRef = useRef<HTMLFormElement>(null);

  async function formAction(formData: FormData) {
    const text = String(formData.get("text") ?? "");
    addOptimistic(text); // ۱) ظاهر می‌شود (با نشانگر «در حال ارسال») UI فوری در
    await sendMessage(formData); // ۲) revalidatePath + درخواست واقعی
    formRef.current?.reset(); // ۳) بعد از پایان React state عوض می‌کند خوش‌بینانه را با داده واقعی عوض می‌کند
  }

  return (
    <div>
      {optimisticMsgs.map((m) => (
        <p key={m.id} className={m.sending ? "opacity-50" : ""}>
          {m.text} {m.sending && <span><img alt="loading spinner" data-bbox="418 448 438 463"/></span>}
        </p>
      ))}
      <form ref={formRef} action={formAction}>
        <input name="text" />
      </form>
    </div>
  );
}
```

تایم‌لاین دقیق چه اتفاقی می‌افتد



نکته‌ی مهم: «برگشت» دستی نیست. اگر اکشن شکست بخورد یا revalidate داده واقعی را بیاورد، چون `optimisticState` از `messages` (prop) مشتق شده، در رندر بعدی به‌طور خودکار به حالت واقعی برمی‌گردد. کار تو فقط یک چیز است: بعد از اکشن، state واقعی را تازه کن (`revalidatePath` در سرور، یا `router.refresh()` یا `setState` با نتیجه).

تله‌ها

- `addOptimistic` را بیرون از `action/transition` صدا زن — به‌روزرسانی خوش‌بینانه فقط در طول همان `action` معنا دارد.

- reducer باید **pure** باشد — `fetch` یا `Math.random` داخلش ممنوع.
- برای id موقت، چیزی یکتا بساز (`crypto.randomUUID()` بهتر از `Date.now`).
- عملیات قابل قبول برای UX خوش‌بینانه = عملیاتی که معمولاً موفق می‌شوند (لایک، ارسال پیام، افزودن به لیست).
حذف حساب بانکی خوش‌بینانه نیست! 😊

۱۳.۵ — `forwardRef` چیست و چرا در React 19 حذف شد؟

قبل از ۱۹: چرا `prop ref` معمولی نبود؟

در React قدیمی، `ref` یک کلید رزرو شده بود — مثل `key` — و React هنگام ساختن props آن را می‌جمع و حذف می‌کرد؛ یعنی هرگز داخل `props` دیده نمی‌شد. دلیل تاریخی: `ref` رفتار مخصوصی داشت (attach به DOM/instance کلاس) و طراحان React خواستند از «آلوده شدن» props جلوگیری کنند.

نتیجه‌ی ناخواسته: **Function Component** نمی‌توانست مستقیم `ref` بگیرد (در کلاس‌ها `ref` یعنی instance، اما برای تابع تعریف ندارد). راه‌حل موقت آن دوره:

```
// روش قدیمی - لازم بود کامپوننت را بیچی ✗
import { forwardRef } from "react";

const MyInput = forwardRef(function MyInput(props, ref) { // ref دوم آرگومان دوم!
  return <input {...props} ref={ref} />;
});

// کابوس می‌شد memo و ترکیب با:
const MemoInput = memo(forwardRef(MyInput));
```

از React 19: `ref` یک `prop` معمولی است

```
// wrapper روش جدید - بدون ✗
function MyInput({ ref, ...props }: {
  ref?: React.Ref<HTMLInputElement>;
} & React.ComponentProps<"input">) {
  return <input ref={ref} {...props} />;
}

// استفاده، هر دو یکی است:
<MyInput ref={inputRef} placeholder="نام" />
```

`forwardRef` از ۱۹ **deprecated** است (هنوز کار می‌کند و کد قدیمی را نمی‌شکند) ولی در کد جدید لازم نیست. حتی می‌توانی `props.ref` را مثل هر `prop` دیگری جابه‌جا، `rename` یا از `spread` عبور دهی.

هدیه‌ی مرتبط در ۱۹ – ref cleanup

تابع‌های callback ref حالا می‌توانند تابع پاک‌سازی برگردانند (قبلا باید در خود callback می‌نوشتی `node = null` تا از باگ leak جلوگیری شود):

```
<input
  ref={node => {
    form.register(node);
    return () => form.unregister(node); // خودکار صدا زده می‌شود detach هنگام
  }}
/>
```

جدول مهاجرت

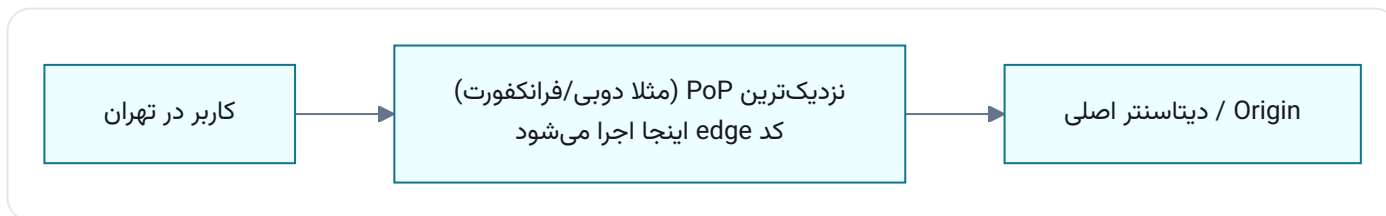
از ۱۹ به بعد	قبل از ۱۹
<code>function C({ ref, ...props })</code>	<code>forwardRef((props, ref) => ...)</code>
<code>memo(C)</code>	<code>memo(forwardRef(C))</code>
<code>ref={() => { ...; return cleanup }}</code>	callback ref بدون cleanup
بدون تغییر – کلاس‌ها همان instance را می‌دهند	Class Component با <code>ref</code>

۱۳.۶ – «Edge» در Middleware یعنی چه؟

کلمه‌ی edge دو معنای مرتبط دارد: یک زیرساخت و یک `runtime`.

۱) Edge Network – زیرساخت

سرویس‌هایی مثل Vercel/Cloudflare، به‌جای یک دیتاسنتر مرکزی، صدها نقطه‌ی کوچک (PoP) دور دنیا دارند. «اجازه» ی شبکه یعنی نزدیک‌ترین نقطه به کاربر:



مزیت: middleware قبل از هر request اجرا می‌شود؛ اگر در دیتاسنتر مرکزی بود، هر کاربر دوردست صدها ms تاخیر اضافه می‌برد. روی edge، فاصله فیزیکی حداقل است و **cold start تقریبا صفر** است (process از قبل گرم است).

۲) Edge Runtime – ساندباکس اجرا

middleware در Next.js (تا قبل از ۱۶) روی Edge Runtime اجرا می‌شد: یک محیط سبک بر پایه‌ی V8 (همان موتور کروم/ورکرها)، شبیه‌ساز Web Worker – نه Node کامل:

روی Edge Runtime ❌	روی Edge Runtime ✅
ماژول‌های Node مثل <code>fs</code> , <code>net</code> , <code>child_process</code>	<code>fetch</code> , <code>Request</code> , <code>Response</code> , <code>URL</code>
پکیج‌های با native binding (مثل <code>bcrypt</code> , <code>prisma</code> , <code>sharp</code>)	Web Crypto, TextEncoder/Decoder, Streams
localStorage/window طبعاً وجود ندارد (سمت سرور است)	<code>cookies/headers</code> و <code>next/server</code>

باینری‌ها کوچک‌اند، شروع سریع است – بهای آن: API محدود. برای کار middleware (چک کوکی/session، ریدایرکت، rewrite، ست هدر) همین‌قدر کافی است.

⚠ تحول Next.js 16 – چون دوره‌ی ما روی Next 16 است، این را بدان

- نام `middleware.ts` به `proxy.ts` تغییر کرده و به‌طور پیش‌فرض روی **Node.js runtime** اجرا می‌شود (دسترسی کامل به API های Node).
- فایل قدیمی `middleware.ts` هنوز پشتیبانی می‌شود و مثل قبل روی edge اجرا می‌شود.
- فلسفه: «middleware naming» خیلی overload شده بود و پروژه‌ها گاهی به API های Node نیاز داشتند؛ `proxy` نقش واقعی‌اش (مرز شبکه) را صادقانه می‌گوید.
- از Next.js 15.5 به بعد حتی روی خود `middleware.ts` هم می‌شد `runtime: export const config = {` گذاشت.

پس جواب مصاحبه‌ای امروزی: «**Middleware** کلاسیک Next یعنی کد مرزی که روی Edge Runtime نزدیک کاربر اجرا می‌شد؛ از Next 16 این لایه `proxy` نام گرفته و روی Node اجرا می‌شود – مفهوم edge حالا بیشتر به زیرساخت CDN/فانکشن‌های لبه مربوط است.» جزئیات در فصل ۱۱.

۱۳.۷ – RSC Payload چیست و کجا سفر می‌کند؟

تعریف

- وقتی یک درخت Server Component روی سرور رندر می‌شود، خروجی‌اش **HTML** نیست – یک قالب **سریال‌شده و قابل استریم** از توصیف UI است به نام **RSC Payload** (پروتکل React برای این کار **Flight** نام دارد). در آن:
- متن/ساختار بخش‌های سرور (مثل HTML اما به‌صورت داده‌ای فشرده)

- ارجاع به chunk های Client Component (کلاینت باید چه JS ای را دانلود و کجا mount کند)
- props سریال شده برای هر Client Component
- ارجاع به Server Action ها و promise های در حال استریم

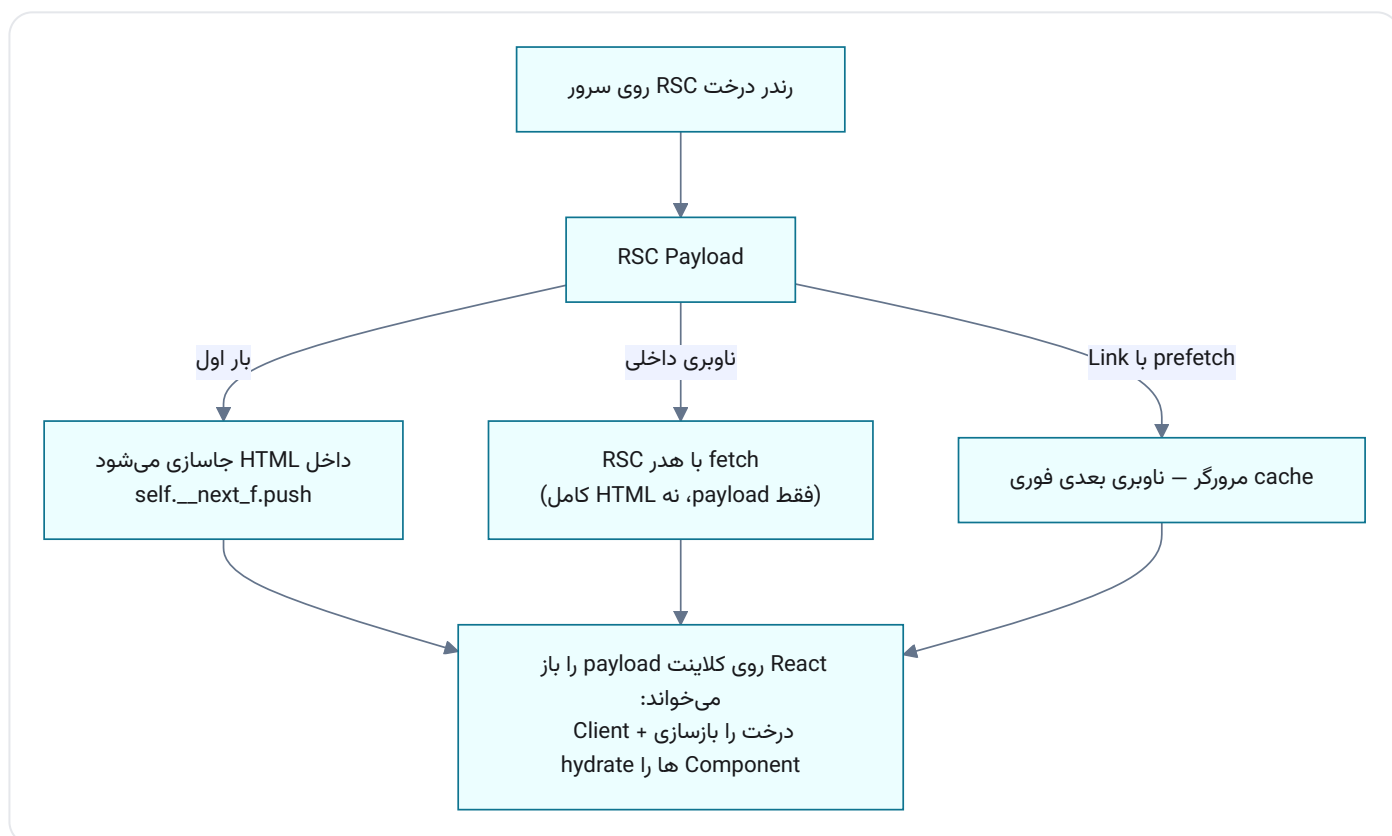
نمونه‌ی واقعی (ساده‌شده)

اگر source صفحه را ببینی، payload را در تکه‌های `self.__next_f.push(...)` می‌بینی:

```
self.__next_f.push([1, '5:["$","div",null,{"className":"card","children":["$","h1",null,{"children"
```

آن "\$" ها و اعداد، marker های پروتکل Flight هستند؛ نوع داده، ارجاع (\$L5 یعنی کامپوننت lazy با شناسه ۵) و props را کد می‌کنند. **JSON خالص نیست** – فرمت مخصوص استریمینگ خودش است.

سه مسیر سفر payload



به همین دلیل ناوبری در App Router سبک است: **layout های مشترک اصلا دوباره نمی‌آیند**، فقط payload مسیر جدید.

«چطوری استفاده میشه؟»

نکته‌ی مهم: مصرف‌کننده‌ی payload، روتر Next.js است – تو مستقیم با آن کار نمی‌کنی. چیزهایی که به تو دسترسی می‌دهند:

- `<Link prefetch>` و `router.prefetch` مسیر را از قبل بگیر

- تازه‌ی همان مسیر را بگیر و merge کن (بدون از دست رفتن client state) `router.refresh()` → payload
- در Server Action → بعد از payload، mutation، تازه در همان پاسخ برمی‌گردد `revalidatePath/revalidateTag` (فصل ۸)

محدودیت‌های ناشی از آن: props ارسالی به Client Component باید قابل سریال‌شدن باشند (فصل ۶) و Server Component ها هیچ JS ای به مرورگر نمی‌فرستند.

۱۳.۸ – Hydration یعنی چه؟

تعریف یک خطی

هیدریشن = تبدیل HTML استاتیک تولیدشده در سرور، به اپ تعاملی React – React روی کلاینت درخت کامپوننت را در حافظه رندر می‌کند، با DOM موجود مقایسه می‌کند و به‌جای ساختن دوباره، به همان گره‌ها `event listener` و `state` وصل می‌کند.

تایم‌لاین

```

کامل را می‌فرستد HTML : سرور
↓
«می‌کند» ← صفحه «دیده می‌شود» ولی «کار نمی‌کند paint را HTML : مرورگر»
↓
دانلود و اجرا می‌شود JS bundle
↓
درخت را در حافظه رندر می‌کند React
↓
Hydration ← موجود تطبیق می‌دهد (نه از صفر!) با DOM
↓
ها وصل می‌شوند ← صفحه تعاملی شد event
  
```

چرا از صفر نمی‌سازد؟ دو دلیل: سریع‌تر است، و state فعلی DOM (مثل متن تایپ‌شده، فوکوس، اسکرول) از بین نمی‌رود.

Hydration Mismatch – ارور کلاسیک

اگر خروجی رندر سرور با رندر اول کلاینت فرق کند، React هنگام تطبیق می‌شکند (در React 19 پیام ارور با diff واقعی DOM نشان داده می‌شود). علت‌های پرتکرار:

```

// ❌ سرور و کلاینت مقدار متفاوتی می‌بینند
function Clock() {
  return <span>{new Date().toLocaleTimeString("fa-IR")}</span>; // هر لحظه فرق دارد
}

// ❌ Math.random() / تاریخ / locale / Intl پیش‌فرض سیستم
// ❌ در بدنه‌ی رندر window/localStorage خواندن
  
```

راه‌حل‌ها:

```
//  بده mount مقدار وابسته به محیط را بعد از
function Clock() {
  const [time, setTime] = useState<string | null>(null);
  useEffect(() => {
    const tick = () => setTime(new Date().toLocaleTimeString("fa-IR"));
    tick();
    const t = setInterval(tick, 1000);
    return () => clearInterval(t);
  }, []);
  return <span suppressHydrationWarning>{time ?? "..."}</span>;
}
```

- **useEffect** : هر چیزی که فقط سمت کلاینت معنا دارد، بعد از mount
- **suppressHydrationWarning** : فقط برای یک المان که تفاوتش اجباری و بی‌خطر است (مثلا ساعت یا افزونه‌ی مرورگر)
- در Server Component، رشته‌ی زمان را یکبار در سرور بساز و بفرست اگر به‌روزرسانی کلاینت نمی‌خواهد

هیدریشن در دنیای RSC – نکته‌ای که مصاحبه دوست دارد

در App Router، «هیدریشن» فقط برای Client Component ها اتفاق می‌افتد. Server Component ها اصلا JS به مرورگر نمی‌فرستند تا هیدریت شوند – خروجی‌شان همان payload است. یعنی **Selective Hydration**: هرچه Client Component کمتر، هیدریشن سبک‌تر. با Suspense هم هر مرز، وقتی آماده شد مستقل hydrate می‌شود.

۱۳.۹ – RPC چیست و چه ربطی به Next.js دارد؟

تعریف عمومی

RPC (Remote Procedure Call) = صدا زدن تابعی که روی ماشین دیگری اجرا می‌شود، طوری که انگار تابع محلی است. لایه‌ی RPC همه‌ی جزئیات را مخفی می‌کند: آدرس HTTP، متد، سریال‌سازی آرگومان‌ها، پاسخ، خطا. تفاوت با REST:

RPC	REST	
عمل/تابع: <code>deletePost(12)</code>	منبع (Resource): <code>/posts/12</code>	واحد
صدا زدن متد	GET/POST/PUT/DELETE	فعل
عملیات دامن‌های، type-safety کامل	API عمومی، CRUD ساده	مناسب

نمونه‌های معروف **gRPC**: RPC: (پروتکل بافر، بین سرویس‌ها)، **trRPC** (RPC تایپ‌سیف در TypeScript)، **JSON-RPC**.

رابطش به دوره: Server Action یک RPC است

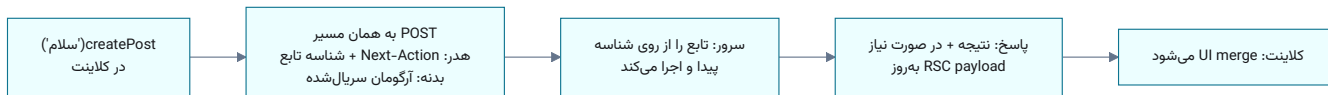
وقتی می‌نویسی:

```
"use server"; // app/actions.ts
export async function createPost(title: string) {
  await db.post.create({ data: { title } });
  revalidatePath("/posts");
}
```

```
"use client"; // هر جا در کلاینت
import { createPost } from "../actions";

<button onClick={() => createPost("سلام")}>...</button>
```

زیر کاپوت این اتفاق می‌افتد:



یعنی کلاینت فقط **reference** تابع را دارد (یک شناسه)؛ بدنه هرگز به مرورگر نرفته است. کل پروتکل انتقالش همان Flight است.

⚠️ امنیت – نکته‌ی مصاحبه‌ای

RPC بودن به معنای «فقط اپ خودم صدا می‌زند» نیست – هر Server Action یک endpoint عمومی HTTP است. مثل هر API باید: ورودی را سمت سرور اعتبارسنجی کنی، احراز هویت/اجازه را داخل اکشن چک کنی، و هرگز به «فرم ما» اعتماد نکنی. (قانون کلی: `use server` یعنی «این تابع URL عمومی دارد».)

کی (RPC) Server Action، کی Route Handler؟

انتخاب	سناریو
Server Action	فرم و mutation های UI خودت
Route Handler	API عمومی برای موبایل/سرویس ثالث
Route Handler	Webhook (Stripe و...)
Route Handler	دانلود/استریم فایل
Server Action + revalidate	mutation + به‌روزرسانی فوری UI در یک پاسخ

✓ جمع‌بندی فصل

- `use` = خواندن Promise (با Suspense) و Context (با شرط)؛ تله‌ی اصلی: promise تازه در هر رندر
- سه‌گانه‌ی فرم‌ها: `useActionState` (نتیجه/ارور/isPending) – `useFormStatus` (pending از فرزند فرم) – `useOptimistic` (UX فوری با بازگشت خودکار)
- `ref` در ۱۹ prop معمولی است؛ `forwardRef` فقط راه‌حل موقت گذشته بود + `cleanup refs`
- Edge = هم زیرساخت (نزدیک‌ترین PoP) هم runtime سبک (نه Node)؛ از Next 16 لایه‌ی middleware به `proxy.ts` روی Node منتقل شده
- RSC Payload = توصیف سریال‌شده‌ی UI (پروتکل Flight)؛ مصرف‌کننده‌اش راوتر Next است؛ ناپذیری = جابجایی payload
- Hydration = چسباندن interactivity به HTML سرور؛ mismatch یعنی رندر سرور ≠ کلاینت
- Server Action = RPC روی پروتکل Flight – و هر RPC یک endpoint عمومی است: اعتبارسنجی اجباری

📝 تمرین فصل ۱۳

1. چرا در `useActionState` امضای اکشن `(prevState, formData)` است ولی Server Action معمولی فقط `formData` می‌گیرد؟ کدام را مستقیم به `<form action>` می‌دهی؟
2. کد زیر چه باگی دارد و چطور درستش می‌کنی؟

```
tsx function Posts() { const posts = use(fetchPosts());  
  // fetchPosts ساده است fetch یک fetchPosts } }
```
3. کامپوننت `<DeleteButton />` را داری که `useFormStatus` صدا می‌زند ولی `pending` همیشه `false` است. دو علت محتمل چیست؟
4. در middleware (یا proxy) چرا نمی‌توانی `bcrypt.compare` صدا بزنی ولی `Web Crypto` را می‌توانی؟ اگر تازه استایل حذف شد، کجای معماری را عوض می‌کنی؟
5. سناریو: بعد از لاگین می‌خواهی کاربر به `/dashboard` برود ولی خود صفحه دوباره لود نشود و header هم به‌روز شود – کدام مکانیزم‌های این فصل در کارند؟ (نام ببر، مرتب کن)

1. چون `useActionState` خودش `prevState` را به اکشن تزریق می‌کند تا نتیجه‌ی اجرای قبلی در دسترس باشد؛ `Server Action` معمولی چنین `state` ای ندارد و فقط `formData` می‌گیرد. نسخه‌ی سوم (خروجی `useActionState` یعنی `formAction`) به `<form action>` داده می‌شود.
2. `promise` در هر رندر تازه ساخته می‌شود → `suspend` بی‌نهایت. درست: `promise` را در `Server Component` بساز و به‌عنوان `prop` بده (و `<Suspense>` بگذار)، یا با `cache()` از `react` بساز.
3. ① هوک بیرون از `<form>` صدا شده (کامپوننت خود `form`، نه فرزندش)؛ ② دکمه با `JS disabled/action` معمولی `submit` می‌شود و `pending` اصلاً فعال نمی‌شود – یا این که فایل از `import react` کرده نه `react-dom`.
4. چون `middleware` کلاسیک روی `Edge Runtime` است – مازول‌های `Node/native` ندارد؛ `Web Crypto` استاندارد وب است و در آن ساندباکس هست. درست: هش/کوکی را با `Web Crypto` (یا کتابخانه سازگار با `edge` مثل `jose`) چک کن، یا در `Next 16` کار سنگین را به `proxy.ts` روی `Node` ببر.
5. ① `Server Action` لاگین → `redirect` ② `redirect("/dashboard")` باعث ناوبری `RSC` می‌شود: فقط `RSC` `Payload` مسیر جدید `fetch` می‌شود (layout مشترک دست نمی‌خورد) ③ در همان پاسخ `payload` به‌روز، `header` و دیگر `Server Component` ها `re-render` می‌شوند ④ اگر دکمه `submit` فرم بود، `pending` با `.useFormStatus/useActionState`.

→ ادامه: چیت‌شیت‌های فصل ۱۲ را با این فصل ترکیب کن – این ۹ موضوع دقیقاً سوال‌های پرتکرار مصاحبه‌اند.